TEMA5: ADMINISTRACIÓN DE APACHE

FUNCIONAMIENTO DE UN SERVIDOR WEB

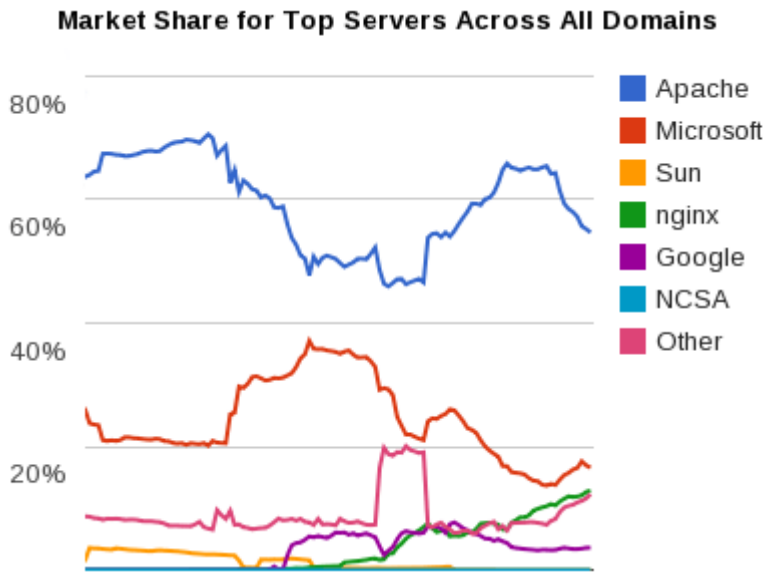
Detrás de cada página web debe existir un servidor web que ofrezca esa página, bien a los internautas, a los trabajadores de una empresa (por tratarse de una página web interna, de la empresa, no accesible a Internet), o a todo aquel que disponga de una conexión de red con la cual pueda acceder a la página.
La configuración del servidor web dependerá de las páginas web que ofrezca, así la configuración no será la misma si la página posee contenido estático o no, o si se necesita que modifique el contenido según interacción del usuario, o si se necesita de comunicación segura en la transición de información, o si se debe tener en cuenta el control de acceso a determinados sitios de la página. Por lo tanto según las páginas web que se ofrezcan el servidor web deberá estar configurado para tal fin: con soporte PHP, con soporte de cifrado, con soporte de control de acceso, etc.

Además, un servidor web puede alojar varios dominios de Internet.
Por todo lo anteriormente comentado veremos cómo configurar y administrar el servidor Apache ya que soporta: páginas web estáticas, dinámicas, hosts virtuales, seguridad mediante cifrado, autenticación y control de acceso, modularización y monitorización de archivos de registro.
Uno de los servidores Web que tiene más historia, funcionalidades y es usado en la mayoría de los sitios web es sin duda ***Apache***. El servidor http *Apache* es uno de los proyectos desarrollados por la Fundación Apache que es una organización no lucrativa creada para dar soporte a los proyectos de software bajo la denominación **Apache**. *Apache Software Foundation* es una comunidad descentralizada de desarrolladores que trabajan cada uno en sus propios proyectos de código abierto (*open source*).

CARACTERÍSTICAS

- Servidor Web potente, flexible y ajustado al HTTP/1.1.
- Modular. Altamente configurable y extensible.
- Servidor "Open Source".
- Multiplataforma: Windows, Linux, Unix, …
- Servidor "mas utilizado" en Internet.

http://www.netcraft.com



INTALACIÓN (DEBIAN/UBUNTU)

Hay dos formas de Instalar Apache:

- Por paquete, es la forma más sencilla.
- Manual, descargando el código fuente y compilándolo con una configuración determinada. Permite afinar mucho más la configuración.

Realizaremos la instalación por paquete por ser la más senciila. Sólo es necesario ejecutar un gestor de paquetes, por ejemplo ***apt-get***, ***aptitude*** o mediante el interfaz gráfico ***Synaptic***. Hay que tener en cuenta que hay que ejecutar todas las ordenes como administrador (root) y si no es así hay, hay que iniciar la orden con precediéndola de la orden ***sudo*** que permite a los usuarios ejecutar programas con los privilegios de seguridad de otro usuario (normalmente el usuario *root*). El archivo de configuración ***/etc/sudoers*** especifica qué usuarios pueden ejecutar qué comandos en nombre de qué otros usuarios. Como ***sudo*** es muy estricto con el formato de este archivo, y cualquier error podría causar problemas serios, existe la utilidad ***visudo***; ésta permite al usuario *root* editar el archivo y luego revisar su corrección antes de guardarlo.
Los pasos que seguiremos (en modo consola) son:

1. Instalación del paquete:

   sudo apt-get update (actualiza la lista de paquetes en los repositorios)

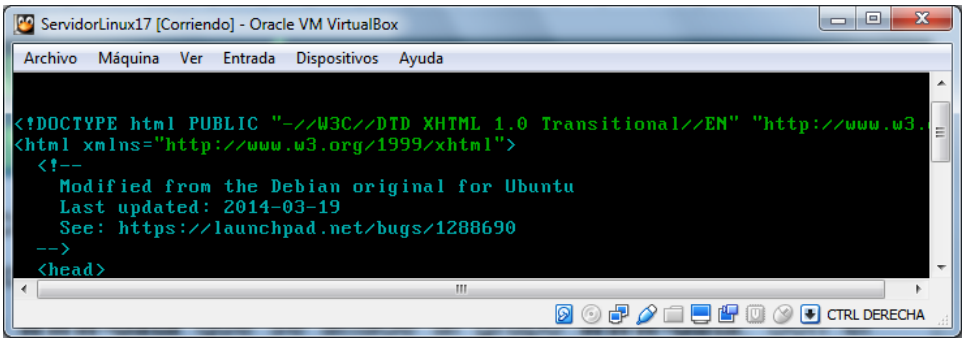
Básicamente un repositorio es un servidor (pagina, ftp, o cualquier lugar en internet) en donde se encuentran los programas específicos para el SO Linux. La mayoría de distribuciones de Linux tienen su lista de repositorios. *La orden **apt-get update** actualiza esta lista con las últimas versiones disponibles de cada uno.*

---

sudo apt-get install apache2

Se crean:

- Los archivos de configuración en */etc/**apache2***.
- El usuario **www-data** que se añade al grupo **www-data**. Son el usuario y el grupo con el que se ejecutan los procesos de Apache que se encargan de atender peticiones.
- El directorio /var/www**/html**. Es el directorio raíz donde se almacenan las páginas web del servidor virtual por defecto. Su propietario es *root* y su grupo es *root*. En esta carpeta ya hay un archivo *index.html*.



Para averiguar la versión de Apache instalada:

apache2 –v

Normalmente, una vez que se ha terminado de descargar el paquete, apache comenzará a funcionar. Para desinstalar apache:

sudo apt-get purge apache2

2. Comprobación del funcionamiento:

   ps –ef | grep apache2

   Con esta orden podemos ver si hay algún proceso en el sistema que esté en ejecución y se llame apache2.

3. Comprobar si el servidor está en disposición de atender las peticiones http. Para ello abrimos el navegador y ponemos:

   http://*Dirección IP del host*

   Con ello estamos lanzando una petición Web desde el propio servidor para que él mismo conteste. Si recibimos un error de conexión, apache no estará funcionando. Si todo ha ido correctamente apache sirve por defecto el fichero index.html (que está en el directorio */var/www/html*) que nos muestra un mensaje anunciando que el servidor está trabajando correctamente.

PARAR/INICIAR APACHE(DEBIAN/UBUNTU)

Cada que vez que cambien los ficheros de configuración para que se apliquen los cambios hay que:

- Parar/iniciar el servidor.
- O indicarle que lea de nuevo los ficheros de configuración.

Para Parar/Arrancar Apache :

- **Inicio (varias posibilidades):**
  - **sudo service apache2 start**
  - **sudo /etc/init.d/ apache2 start**
  - **sudo apachectl start**

- **Parada (varias posibilidades):**
  - **sudo service apache2 stop**
  - **sudo /etc/init.d/ apache2 stop**
  - **sudo apachectl stop**

- **Reinicio (varias posibilidades):**
  - **sudo service apache2 restart**
  - **sudo /etc/init.d/ apache2 restart**
  - **sudo apachectl restart**

- **Leer de nuevo los ficheros (varias posibilidades):**
  - **sudo service apache2 reload**
  - **sudo /etc/init.d/ apache2 reload**

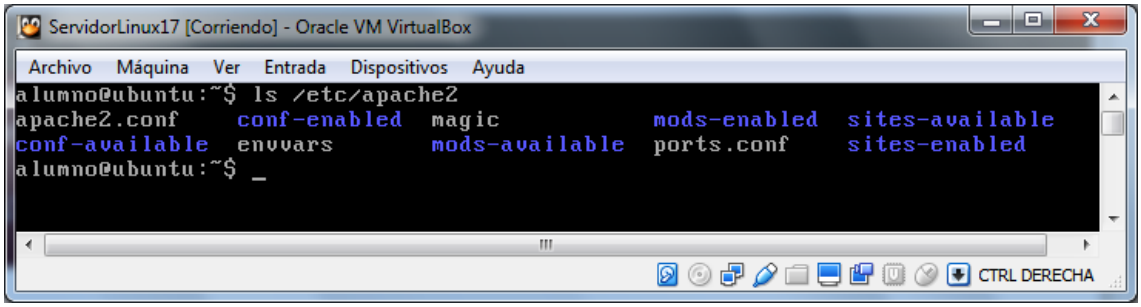
SERVIDOR "PRINCIPAL" Y SERVIDORES VIRTUALES

- **Linux (Debian/Ubuntu):**
  - La versión precompilada para Debian/Ubuntu tiene habilitados servidores virtuales.

- Tiene creado y habilitado un servidor virtual por defecto (default).

FICHEROS DE CONFIGURACIÓN (DEBIAN/UBUNTU)

Una vez instalado *Apache*, es necesario configurarlo para que se ajuste a las necesidades específicas del proyecto que se tiene en mente.
La configuración de *Apache* queda determinada por un conjunto de directivas que se escriben en los ficheros de configuración. Dependiendo del entorno en el que se escriban determinadas directivas de configuración, estas afectarán al comportamiento del servidor en general, aun servidor virtual, a un directorio en concreto, a un fichero, etc.
Existen directivas (por ejemplo <VirtualHost></VirtualHost>, <Directory></Directory>, …) que actúan como contenedores de otras directivas.
Los archivos de configuración de apache2 contienen las llamadas directivas de configuración y se encuentran en la carpeta */etc/apache2*.



*/etc/apache2/**apache2.conf***

El fichero de configuración principal en este tipo de instalación es apache2.conf y se encuentra en ***/etc/apache2/apache2.conf*** e incluye las directivas principales del servidor. Al final del archivo y mediante órdenes *include* se le indica a Apache que debe cargar otros archivos donde se encuentra la configuración de otros componentes.
Este fichero se puede dividir en tres secciones (las directivas pueden aparecer mezcladas, desordenadas o incluidas en otros ficheros de configuración; esta organización es a nivel puramente organizativo, no es una división estricta):

- Sección 1: Directivas globales. Por ejemplo: el número máximo de clientes concurrentes, los *timeouts*, el directorio donde están los ficheros de configuración, etc.
- Sección 2: Directivas de funcionamiento del servidor principal. Agrupa las directivas que definen el comportamiento del servidor principal. También definen el comportamiento por defecto de todos los servidores virtuales que se configuren (incluido un posible servidor virtual por defecto).
- Sección 3: Directivas de configuración de los servidores virtuales. Son directivas relacionadas con los servidores virtuales que se definan.

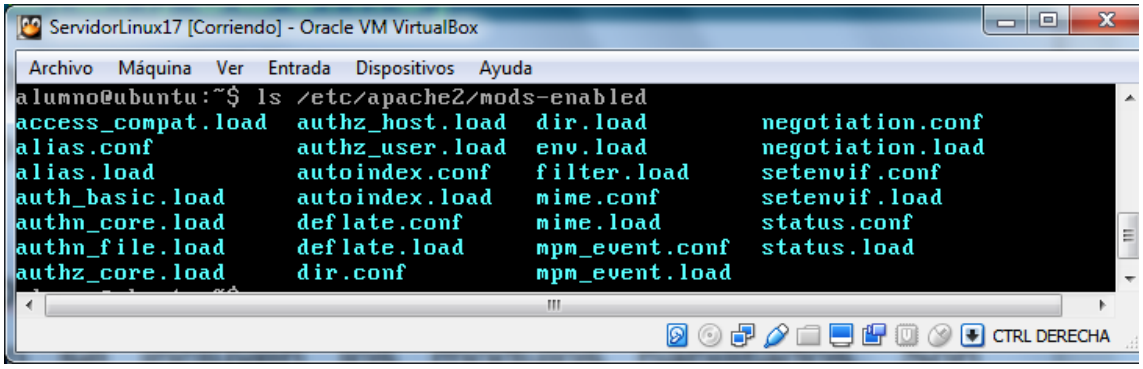
```
# Formato general apache2.conf

# Aquí empieza la Sección 1 (directivas de configuración global)

… Directivas globales

# Aquí empezaría la sección 2 (directivas de funcionamiento del
# servidor principal )
… Directivas de funcionamiento del servidor principal (se heredan en
los servidores virtuales)
User ${{APACHE_RUN_USER}
Group ${{APACHE_RUN_GROUP}

include /etc/apache2/mods-enabled/*.load
include /etc/apache2/mods-enabled/*.conf
include /etc/apache2/httpd.conf
include /etc/apache2/ports.conf

.. Directivas de logs y errores
include /etc/apache2/conf.d/

# Aquí empezaría la sección 3 (Servidores virtuales)
include /etc/apache2/sites-enabled/
```

/etc/apache2/**ports.conf**
En este fichero se establecen las direcciones IP y los puertos que va a usar el servidor. En la instalación que hemos llevado a cabo se establece que se escuche en el puerto 80 (http por defecto) y en caso de que activemos el protocolo SSL se usaría el puerto 443.
También se habilitan los servidores virtuales (directiva **NameVirtualHost *:80**) para todas las peticiones realizadas desde todas las Ips (*) por el puerto 80.

/etc/apache2/**envvars**
Se utiliza para configurar las variables de entorno del servidor.

/etc/apache2/mods-available/
En este directorio se incluyen los módulos disponibles.

/etc/apache2/mods-enabled/
En este directorio se incluyen los módulos habilitados. Son realmente enlaces simbólicos a los ficheros de *mods-available*.



/etc/apache2/sites-available/
En este directorio se incluyen el fichero **000-default.conf** con la configuración del servidor virtual por defecto.

```
alumno@ubuntu:/etc/apache2/sites-available$ ls
000-default.conf   default-ssl.conf
```

/etc/apache2/sites-enabled/
Contiene el fichero **000-default** que es un enlace al fichero *000-default .conf* del directorio *sites-available*.

```
alumno@ServidorLinux01:/etc/apache2/sites-enabled$ ls
000-default
```

Además, en este archivo *apache2.conf* se pueden añadir otros archivos de configuración mediante la directiva **include**. Al final del archivo aparece una sección de inclusiones que permiten subdividir la configuración del servidor Apache como mejor nos parezca. En este caso se muestran:

```
# Include module configuration:
Include mods-enabled/*.load
Include mods-enabled/*.conf

# Include the virtual host configurations:
Includesites-enabled/
```

Include sites-enabled se usa para crear en el directorio /etc/apache2/sites-*enabled* los ficheros de configuración de los sitios virtuales que veremos más adelante.
Si nuestro servidor es pequeño quizá nos baste con usar un único fichero de configuración con todas las directivas pero en cuanto vaya a ser un poco complejo nos irá mejor si lo dividimos siguiendo algún criterio lógico.

DIRECTIVAS

Los archivos de configuración están compuestos por directivas que le indican al servidor cómo actuar.
Las directivas se pueden consultar en:

http://httpd.apache.org/docs/2.2/mod/directives.html

A continuación se muestran algunas directivas de configuración importantes.
En primer lugar destacamos directivas **globales**:

Port
Nos permite cambiar el puerto en el que el servidor espera las peticiones estándar.

```
Port 7000
```

ServerRoot
Directorio raíz de la instalación de Apache.

TimeOut
Tiempo máximo de espera que espera el servidor la respuesta del cliente antes de cortar la conexión TCP. Por defecto es 300 segundos y se recomienda no cambiarlo.

```
Timeout 300
```

KeepAlive
Determina si el servidor va a permitir que cada conexión haga más de una petición, es decir si la conexión TCP/IP es persistente. El problema de activarlo es que un único cliente puede consumir demasiados recursos y saturar el servidor por lo que

en caso de establecerlo a *on* se recomienda configurar cuidadosamente *KeepAliveTimeout*, generalmente a un nivel bajo.

```
KeepAlive On
```

MaxKeepAliveRequests
Determina el número de peticiones que podrá realizar cada conexión. Evidentemente solo tiene sentido si *KeepAlive* está activada.

```
MaxKeepAliveRequests 100
```

KeepAliveTimeout
Determina el tiempo que el servidor esperará antes de atender una nueva petición del mismo cliente en la misma conexión.

```
KeepAliveTimeout 15
```

User y **Group**
Estas dos directivas indican con qué usuario y grupo se ejecuta el servidor. No debe ser elusuario *root*por razones de seguridad ya que crearía una brecha perfecta para los hackers.
En Linux por defecto el usuario *nobody* y el grupo *nogroup* tienen muy pocos privilegios por lo que son buenos candidatos.
Si queremos usar el número del grupo o del usuario en lugar del nombre debemos añadir # justo antes del número.

```
Usernobody
Group #-1
```

ServerName
Nombre del servidor web. Es el nombre que se escribe en el navegador para solicitar la página. Por ejemplo:

```
ServerName www.ciudadescolar.com
```

DocumentRoot
Directorio donde se almacenan las páginas del sitio web.

```
DocumentRoot "/var/www/html"
```

UserDir
Indica si se debe permitir que cada usuario de nuestro sistema tenga su propia carpeta personal en el servidor web y establece cuál será la ruta desde el servidor para acceder a dicha carpeta. Esto tiene sentido por ejemplo si se monta un servidor Apache en un instituto y cada profesor tiene su propio usuario. Para que cada uno tenga su propia web personal se puede activar esta opción.

```
UserDir  public_html
```

Pero la opción que se activa por defecto es la que hace que los usuarios no tengan su propio espacio.

```
UserDir  disable
```

En caso de activar los directorios de usuarios se recomienda deshabilitar el de *root*

```
Userdir disabled root
```

DirectoryIndex
Se especifican los archivos por defecto que se sirven desde el servidor. El valor predeterminado es (se establece en el fichero ***/etc/apache2/mods-enabled/dir.conf***)

```
DirectoryIndex index.html index.htm default.htm index.php index.pl
```

Si accedemos a un directorio que no contiene ninguno de los archivos especificados, Apache crea dinámicamente un archivo que lista los contenidos.

AccessFileName
Indica el nombre del archivo en el que se deben buscar las directivas de acceso determinadas en cada directorio. Por defecto es *.htaccess* y no se recomienda cambiarlo en absoluto.

```
AccessFileName .htaccess
```

ErrorLog
Esta directiva es muy importante ya que indica dónde ubicar el archivo de registro de los errores que se produzcan en el servidor. El lugar por defecto es */var/log/apache2/error.log*. En el fichero *apache2.conf* se establece utilizando una variable de entorno definida en el fichero ***/etc/apache2/envars***.

```
ErrorLog ${APACHE_LOG_DIR}/error.log
```

LogLevel
Establece cuánta información se guardará en el archivo de registro de errores (nivel de verbosidad),es decir si la descripción de los errores será más o menos detallada. El nivel por defecto es suficiente para empezar pero cuanto mayor sea el servidor y más importante su función más información necesitaremos. Los valores posibles incluyen: *debug, info, notice, warn, error, crit, alert, emerg*.

```
LogLevel  warn
```

ServerSignature
Indica si al mostrarse una página generada automáticamente por el servidor (no las generadas mediante lenguajes a partir de algo establecido por el usuario sino las páginas de error, listado de directorios FTP, etc) debe mostrarse el nombre y la versión del servidor. Esto puede ser usado maliciosamente así que si no estás conforme establécelo a off.

```
ServerSignature  On
```

LogFormat
Establece qué y en qué formato se registrará el fichero *log* de accesos.

CustomLog
Establece la ruta al archivo que registra las visitas a nuestro sitio web.

```
CustomLog ${APACHE_LOG}/access.log combined
```

Combined indica el uso de un único archivo para guardar toda la información ya que es posible dividir este registro en varios.

AddLanguage
Sirve para asociar extensiones a idiomas determinados para el contenido. Esta directriz es útil para la negociación de contenidos entre el servidor y el navegador web del cliente ya que Apache puede devolver contenidos en diferentes idiomas dependiendo de la configuración del idioma del navegador Web. Es útil sobre todo en los casos en los que tenemos el sitio escrito en varios idiomas ya que permitirá que Apache sirva la adecuada en cada caso en función de la configuración del navegador del cliente. Esto se realiza de forma transparente para el usuario final.
Los códigos de idiomas se definen en la especificación ISO 639. Más concretamente en la ISO 639-1 los códigos de dos letras y en la ISO 639-2 los de tres letras que incluyen más idiomas.

```
# Danés (da) - Holandés (nl) - Inglés (en) - Estonio (et)
# Francés (fr) - Alemán (de) – Griego Moderno (el)
# Italiano (it) - Noruego (no) - Koreano (kr)
# Portugués (pt) - Luxemburgués(ltz)
# Español (es) - Sueco (sv) - Checo(cz)
# Polaco (pl) - Portugués de Brasil (pt-br) - Japonés (ja)
# Ruso (ru) - Croata (hr)
#
AddLanguage es .es
AddLanguage da .dk
AddLanguagenl .nl
AddLanguage en .en
AddLanguageet .et
AddLanguagefr .fr
AddLanguage de .de
AddLanguage el .el
AddLanguage it .it
AddLanguageja .ja
AddLanguagepl .po
AddLanguagekr .kr
AddLanguagept .pt
AddLanguage no .no
AddLanguagept-br .pt-br
AddLanguageltz .ltz
AddLanguagesv .se
AddLanguagecz .cz
AddLanguageru .ru
AddLanguagetw .tw
AddLanguagehr .hr
```

Ten en cuenta que el indicador del idioma no tiene por qué ser idéntico al sufijo como se ve en varios de los ejemplos.

LanguagePriority
Permite establecer una prioridad de los idiomas en caso de que no se especifique uno o haya un empate en la negociación por diferentes motivos. Por defecto viene en inglés, pero en la mayoría de los casos nosotros querremos establecerlo en español.

```
LanguagePriority es en fr de
```

AddCharset
Es igual que *AddLanguage* pero para añadir nuevos juegos de caracteres.

```
AddCharset ISO-8859-1 .iso8859-1 .latin1
AddCharset ISO-8859-2 .iso8859-2 .latin2 .cen
```

AddDefaultCharset
Debería establecerse al juego de caracteres que mejor se ajuste a la zona en la que se sitúa el servidor y al idioma del contenido. En caso de no estar seguros es mejor dejarlo como está.

```
AddDefaultCharset ISO-8859-1
```

ErrorDocument 404
Establece el mensaje que se visualiza cuando no se encuentra la página y no se muestra el índice de ficheros del sitio web. También puede especificarse la visualización de una página. Ejemplos:

ErrorDocument 404 "Página no encontrada"
ErrorDocument 404 /404.html

LoadModule
Cuando instalamos Apache, habilitamos la opción de *Dynamic Shared Object* (DSO) que permite añadir módulos dinámicamente sin necesidad de recompilar el servidor. La directiva *LoadModule* indica qué módulos dinámicos cargar. No es necesario incluir los módulos que se compilaron con el servidor.

UseCanonicalName
Indica a Apache que cada vez que tenga que construir direcciones que hagan referencia a si mismo lo haga con el nombre canónico (el indicado en la directriz *ServerName*) y el puerto en lugar del nombre de la máquina (hostname) y el puerto.

```
UseCanonicalName  On
```

TypesConfig
En el servidor web Apache existe el archivo **/etc/apache2/mods-available/mime.conf** donde se encontra una referencia al archivo **/etc/mime.types**, el cual contiene la lista de tipos MIME reconocidos por el servidor.

Esta directiva indica dónde estará el archivo que describe los tipos MIME (mime.types).

TypesConfig ***path*/mime.types**

DefaultType
Establece el tipo MIME para todos aquellos archivos a los que no se les pueda asignar uno mediante su extensión, etc. Si tenemos un servidor donde la mayoría del contenido son páginas HTML, XML, etc es buena idea usar el valor por defecto, pero si la mayoría del contenido son archivos binarios(fotografías, programas, etc) sería conveniente establecer *application/octet-stream*.

```
DefaultType text/html
```

ForceType hace que todos los ficheros cuyos nombres tengan una equivalencia con lo que se especifique sean servidos como contenido del tipo MIME que se establezca.

```
ForceType image/gif
```

Desplegará el contenido como un archivo de imagen *gif*.

Directory
Esta opción se usa para configurar cómo se comportará y qué se permitirá en cada directorio al que tiene acceso el servidor Apache. Esta configuración se aplica a un directorio y los subdirectorios que contiene si no se sobrescribe en otra definición sobre un directorio más concreto.

Otro grupo de directivas son las asociadas con ***Directory***:

DirectoryIndex
Establece que el fichero *prueba.html* se sirve por defecto en el caso que de que al conectarse al servidor no se solicite ningún recurso en concreto.

AllowOverride
Permite que las opciones de configuración definidas previamente puedan ser sobrescritas para este sitio a través de ficheros **.htaccess** (all, None, ...)

Options
Opciones disponibles en el directorio:

FollowSymLinks	Permite seguir enlaces simbólicos
Indexes	Si en el directorio del sitio web no está el fichero por defecto, visualiza lista de archivos del directorio. Para lo contrario: *-indexes*

MODULOS

Hay muchas maneras de instalar Apache. Nosotros hemos usado una que utiliza DSO (Dynamic Shared Objects). Esto es una configuración de Apache que permite añadir luego módulos sin necesidad de recompilar todo el servidor.
El diseño de Apache es modular. El núcleo de Apache incluye la funcionalidad necesaria para establecer un servidor web pero existen muchos módulos adicionales que permiten añadir funciones extra.
El uso de DSO tiene unas ventajas evidentes y la única desventaja es una pequeña disminución del rendimiento del servidor por lo que se recomienda su utilización excepto en casos en los que el rendimiento sea crítico.
Cada módulo tiene un conjunto de directivas específicas que permiten su gestión.

Tipos

- **Módulos estáticos** que se añaden cuando se compila Apache.
- **Módulos que se cargan dinámicamente** cuando se inicia el servidor. Hay que compilar el servidor con la opción DSO. Es el que nosotros usaremos.
  Ventajas DSO:
  - Servidor más flexible.

  Desventajas DSO:
  - Servidor es más lento en el arranque.
  - Servidor más lento en funcionamiento.

Directivas

- **LoadModule**

Permite cargar módulos dinámicos.

- ***<IfModulenombre_modulo> … </IfModule>***

Permite especificar directivas que se tendrán en cuenta si el módulo está cargado.

```
LoadModule dir_module /usr/lib/apache2/modules/mod_dir.so
```

```
<IfModule mod_dir.c>

        DirectoryIndex index.html index.cgi index.pl index.php index.xhtml

</IfModule>
```

MÓDULOS (Debian/Ubuntu)

Módulos disponibles

Podemos encontrar los módulos disponibles para cargar en la carpeta ***/usr/lib/apache2/modules*** (archivos **.so**):

ls -la /usr/lib/apache2/modules

```
alumno@ServidorLinux01:/usr/lib/apache2/modules$ ls
httpd.exp                mod_cgi.so             mod_mime_magic.so
mod_actions.so           mod_charset_lite.so    mod_mime.so
mod_alias.so             mod_dav_fs.so          mod_negotiation.so
mod_asis.so              mod_dav_lock.so        mod_proxy_ajp.so
mod_auth_basic.so        mod_dav.so             mod_proxy_balancer.so
mod_auth_digest.so       mod_dbd.so             mod_proxy_connect.so
mod_authn_alias.so       mod_deflate.so         mod_proxy_ftp.so
mod_authn_anon.so        mod_dir.so             mod_proxy_http.so
mod_authn_dbd.so         mod_disk_cache.so      mod_proxy_scgi.so
mod_authn_dbm.so         mod_dumpio.so          mod_proxy.so
mod_authn_default.so     mod_env.so             mod_reqtimeout.so
mod_authn_file.so        mod_expires.so         mod_rewrite.so
mod_authnz_ldap.so       mod_ext_filter.so      mod_setenvif.so
mod_authz_dbm.so         mod_file_cache.so      mod_speling.so
mod_authz_default.so     mod_filter.so          mod_ssl.so
mod_authz_groupfile.so   mod_headers.so         mod_status.so
mod_authz_host.so        mod_ident.so           mod_substitute.so
mod_authz_owner.so       mod_imagemap.so        mod_suexec.so
mod_authz_user.so        mod_include.so         mod_unique_id.so
mod_autoindex.so         mod_info.so            mod_userdir.so
mod_cache.so             mod_ldap.so            mod_usertrack.so
mod_cern_meta.so         mod_log_forensic.so    mod_version.so
mod_cgid.so              mod_mem_cache.so       mod_vhost_alias.so
```

Puedes consultar los módulos estáticos que se han cargado al compilar el servidor mediante el comando:

apache2ctl –l

Otra forma de consultarlo es:

apache2ctl –M

que nos muestra los módulos por nombre y determinando si se añadieron de forma estática (cuando se compiló Apache) o de manera dinámica (al arrancar el servidor). Estos últimos se pueden averiguar también consultando el directorio **/etc/apache2/mods-enabled**. Podemos observar que los ficheros que aparecen

en este directorio son enlaces simbólicos a ficheros de **/etc/apache2/mods/available**.

Activación de módulos

Para activar un módulo de Apache hay que usar la directiva ***LoadModule*** del módulo *mod_so*. Cuya sintaxis viene definida como

sudo LoadModule *module filename*

Donde *module* es el nombre del módulo y *filename* la ruta y nombre del archivo *.so* dónde se encuentra.
Por ejemplo para activar el módulo *mod_speling*:

```
sudo LoadModule speling_module usr/lib/apache2/modules/mod_speling.so
```

Siendo necesario después reiniciar Apache:

sudo/etc/init.d/apache2 restart

En algunas distribuciones de Linux, como es nuestro caso con Ubuntu, hay otra forma de activar los módulos. Si nos damos cuenta tenemos dos *directorios /etc/apache2/mods-available* y */etc/apache2/mods-enabled*. En el primero están todos los módulos que vienen incluidos en esta versión de Apache mientras que en el segundo están los que tenemos activos en nuestro servidor mediante enlaces simbólicos a los módulos en el directorio anterior.
Para activar un módulo usamos el comando ***a2enmod*** con el nombre del módulo.

```
sudo a2enmod speling
```

que nos informa de lo que está sucediendo y de que es necesario reiniciar el servidor.
y para desactivarlo se hace igual pero con el comando ***a2dismod***.

```
sudo a2dismod speling
```

después de cada activación o desactivación es necesario reiniciar el servidor.

sudo/etc/init.d/apache2 restart

Uso y configuración de módulos

Cada módulo en Apache2 tiene un uso y configuración diferentes. Para saber cómo usar cada módulo lo mejor es consultar la documentación de la página oficial. Desde la lista de módulos se puede hacer click en cada uno para ver sus directivas y uso.
Nosotros vamos a seguir con el ejemplo del módulo *mod_speling* que solo tiene dos directivas que podemos usar: ***CheckSpelling*** y ***CheckCaseOnly***. La primera activa o desactiva el módulo y la segunda hace que solo se corrijan los errores de mayúsculas/minúsculas. Para activar la corrección completa debemos añadir al archivo de configuración la siguiente línea:

CheckSpelling on

Después de cada cambio en la configuración es necesario reiniciar la máquina para que la modificación surta efecto.

Sudo /etc/init.d/apache2 restart

Para probar si todo funciona correctamente podemos intentar cargar la página web: ***http://localhost/insdex.html*** antes de activar el módulo en el servidor. Nos devuelve una página de error generada automáticamente con Apache con el siguiente texto:

```
Not Found
The requested URL /insdex.html was not found on this server.
Apache/2.2.22 (Ubuntu) Server at localhost Port 80
```

Si después de activar el módulo intentamos acceder a la misma página web veremos que corrige la dirección a *http://localhost/index.html* y carga la página de inicio de Apache.

Archivos de configuración asociados a cada módulo

En el directorio ***/etc/apache2/mods-enabled*** podemos ver que hay dos archivos asociados a cada módulo dinámico activo. Los archivos ***.load*** incluyen la directiva para que se cargue el módulo en cuestión mientras que los archivos ***.conf*** incluyen directivas de configuración que únicamente se aplicarán si se carga un módulo determinado. Esto se consigue con la directiva *IfModule* que funciona como un *if* de programación.
Realmente lo que encontramos en el directorio son enlaces simbólicos a los archivos en */etc/apache2/mods-available*.

mod_status y mod_info

Vamos a comprobar los archivos de configuración activando los módulos *mod_status y mod_info.* Estos dos módulos nos permiten obtener información muy útil sobre nuestro servidor.

mod_status está activado por defecto en nuestra distribución de Apache, pero se activa y desactiva como cualquier otro módulo. Podemos ver y modificar su configuración en */etc/apache2/mods-available/*:

sudo nano/etc/apache2/mods-available/status.conf

Vamos a aprovechar para ver otra directiva de configuración. Si nos fijamos ahora solo nos permite consultar el estado desde nuestra máquina.

Allow from 127.0.0.1 ::1

Podemos añadir una dirección IP de otra máquina para que nos dé acceso desde ella.

Allow from 127.0.0.1 ::192.168.37.45

Se consulta dicha información en la dirección o nombre de nuestro servidor con:

http://192.168.37.45/server-status (si nuestra dirección fuera esa)

También se puede hacer desde el terminal:

sudo apache2ctl status

mod_info no viene activado por defecto, por lo que hay que activarlo:

sudo a2enmod info

podemos consultar o modificar su configuración:

```
sudo nano /etc/apache2/mods-enabled/info.conf
```

Esta información se puede consultar en ***http://192.168.37.45/server-info*** modificando la dirección IP por la tuya o el nombre de tu servidor.
Recuerda que para poder usar los cambios en la configuración hay que reiniciar el servidor.

Espacio web para cada usuario (módulo *userdir*)

En determinadas circunstancias (por ejemplo una institución educativa como un instituto o una universidad) es útil que cada usuario tenga un directorio en el que pueda crear su propio conjunto de páginas web. Si el número de usuarios es grande, esto puede cargar innecesariamente de trabajo al administrador del servidor. Apache ofrece una alternativa automatizada para esta situación mediante el módulo ***userdir***.
Si activamos el módulo (por defecto no viene activo) cada usuario tendrá un espacio al que se accederá mediante la URL*http://sitioejemplo.com/~nombreusuario* donde *nombreusuario* será el nombre que tiene en el servidor Linux.
Cada usuario puede crear ahora en su carpeta personal un directorio *public_html* donde ubicar sus páginas. Aquí se explica en más detalle cómo se utiliza la carpeta personal.
No se activa esta opción para el usuario *root*.
Cada usuario del sistema dispondrá de un espacio web que se almacena dentro de su carpeta *home* en una carpeta llamada *'public_html'* (por ejemplo, */home/Javier/public_html*). Si dicha carpeta no existe, el propio usuario puede crearla y copiar dentro de ella su página web. Los permisos recomendados son 644, para que el 'grupo' y el 'resto' de usuarios tengan acceso de lectura y así se puedan visualizar las páginas.
El carácter '**~**'comúnmente conocido como gusanillo y que se obtiene con las teclas *Alt Gr + 4* sirve para indicar a apache que debe servir la página desde el home del usuario (en Linux el 'gusanillo' equivale a la carpeta *home*). Por ejemplo, si hemos creado un usuario *javier* y éste ha creado la carpeta */home/javier/public_html* y ha copiado en ella su página web, desde cualquier PC de la red podremos acceder a dicha carpeta yendo a la dirección *http://ip-del-servidor/~javier/*. Para que la página aparezca automáticamente, es necesario crear un archivo llamado *index.html*.

Pasos a seguir:

1. El usuario debe crear en su *home* la carpeta *public_html*:

   mkdir public_html

2. Crear enlaces simbólicos de los siguientes ficheros del módulo *userdir*:

/etc/apache2/mods-available/userdir.conf
/etc/apache2/mods-available/userdir.load

en el directorio ***etc/apache2/mods-enable***. De esta forma los enlaces en este directorio apuntan a las configuraciones guardadas en estos ficheros del directorio */etc/apache2/mods-available* indicando que están activas (hay una directiva *include mods-enabled* en apache2.conf). Para ello usamos el siguiente comando:

sudo a2enmod userdir

Finalmente, reiniciamos el servidor:

sudo /etc/init.d/apache2 restart

El contenido de */etc/apache2/mods-available/**userdir.conf*** es:

```
<IfModule mod_userdir.c>
        UserDir public_html
        UserDir disabled root

        <Directory /home/*/public_html>
                AllowOverride FileInfo AuthConfig Limit Indexes
                Options MultiViews Indexes SymLinksIfOwnerMatch IncludesNoExe
                <Limit GET POST OPTIONS>
                        Require all granted
                </Limit>
                <LimitExcept GET POST OPTIONS>
                        Require all denied
                </LimitExcept>
        </Directory>
</IfModule>
```

Con la directiva ***UserDIR*** se establece public_html como directorio personal para todos los usuarios excepto para el usuario *root*. Y con la directiva ***Directory*** se establece la configuración de este directorio. Para comprobar que funciona, desde el navegador:

192.168.37.XX/~usuario

CONTROL DE ACCESO POR HOST (IP/nombre_dominio)

Apache permite restringir algunas zonas del sitio web, en base a la dirección del visitante, esto puede ser realizado de manera fácil siempre que esté cargado el módulo mod_authz_host. Se utiliza la directiva **Require**. Ejemplos:

- **Permitir el acceso sin restricciones:**

```
<Directory /var/www/html/profesor>
        Options Indexes FollowSymLinks
        AllowOverride None
        Require all granted
<Directory>
```

- **Denegar todo el acceso:**

```
Require all denied
```

- **Permitir una única dirección IP:**

```
Require all denied
Require ip 192.168.37.80
```

- La Denegar el acceso a un host:

```
Require all granted
Require not host server01.ejemplo.com
```

- Denegar el acceso a un dominio completo:

```
Require all granted
Require not host ejemplo.com
```

La directiva **Require** puede combinarse mediante las directivas **<RequireAll>...</RequireAll>**, **<RequireAny>...</RequireAny>** y **<RequireNone>...</RequireNone>** para formar reglas más complejas.
<RequireAll>...</RequireAll> indica que la autenticación se cumple si todas las directivas se cumplen. Ejemplo:

```
<RequireAll>
        Require not ip 192.168.100.0/24
        Require not ip 147.156.0.0/16
</RequireAll>
```

Permitirá el acceso si el ordenador no es de la redes 147.156.0.0. y 192.168.100.0
<RequireAny>...</RequireAny> indica que la autenticación se cumple si al menos una de las directivas se cumple. Está establecida por defecto. Ejemplo:

```
<RequireAny>
        Require ip 192.168.100.8
        Require ip 192.168.100.9
</RequireAny>
```

Permitirá el acceso a las direcciones IP 192.168.100.8 y 192.168.100.9.
<RequireNone>...</RequireNone> indica que la autenticación se cumple si todas las directivas no se cumplen. Ejemplo:

```
<RequireNone>
        Require not ip 192.168.100.8
        Require not ip 192.168.100.9
</RequireNone>
```

Permitirá el acceso todas las direcciones IP escepto 192.168.100.8 y 192.168.100.9.

Autenticación

La autenticación es el proceso para verificar que alguien es realmente quién dice ser.
Muchas veces los sitios web contienen información sensible y dirigida a un grupo reducido de usuarios. Apache se puede configurar para controlar los usuarios que tienen acceso a una página web o a una parte de esta.
Existen módulos que permiten la autenticación.
Se pueden utilizar diferentes sistemas para almacenar la información necesaria: ficheros, bases de datos, servidores de directorios (LDAP).
Pueden utilizarse dos métodos: **Basic** y **Digest**. El primero es más básico y menos seguro que el segundo.
Por ejemplo podemos establecer un directorio al que haya que acceder con contraseña. Para ello es necesario:

- Verificar que los módulos ***auth_basic*** (autentificación *basic*) o ***auth_digest*** (autentificación *digest*) están activados.

  ls –l /etc/apache2/mods-enabled

Si no es así, activar el módulo correspondiente:
sudo a2enmod auth_basic
o
sudo a2enmod auth_digest

sudo /etc/init.d/apache2 restart

- Crear el archivo de contraseñas.
- Configurar el servidor para que solicite un usuario y su contraseña.

El archivo de contraseñas

Autentificación HTTP *Basic*

La contraseña es enviada por el cliente en texto plano. La información sobre el usuario se almacena en un fichero de texto. Es recomendable utilizar este método combinado con conexión SSL (HTTPS).
Es muy importante que este archivo se encuentre en una ubicación que no sea accesible mediante la web. Si el directorio donde se encuentran los ficheros web es**/var/www**, sería buena idea colocar el archivo de claves en una ubicación como /etc/apache2/usuarios/**password**.
La creación del archivo de contraseñas se hace con la utilidad **htpasswd** que viene con la instalación de apache.

sudo htpasswd -c /etc/apache2/usuarios/password *user_name*

donde *user_name*es el nombre del usuario al que daremos autorización.

La aplicación solicitará la contraseña y su confirmación, esta será guardada en el archivo (en este caso, de nombre *password*) en un formato cifrado.
La opción -c indica que se está creando un archivo nuevo, en el caso de que se quieran añadir más usuarios basta con ejecutar en mismo comando pero sin la opción -c, como sigue:

sudo htpasswd /etc/apache2/usuarios/password *user_name*

Autentificación HTTP *Digest*

La contraseña se envía cifrada (¡¡ cifrado débil, no es seguro !!) por el cliente. La información se almacena en un fichero de texto que se crea con la siguiente orden:

sudo htdigest –c /etc/apache2/usuarios/digest *dominio user_name*

Configurar el servidor

El siguiente paso es configurar el servidor para que solicite la contraseña cuando se intente acceder a la página.
Las directivas que permitirán configurar los métodos de autenticación necesitarán ir en el archivo principal del *Virtual host* (en una sección **<Directory>**) o en un fichero *.htaccess* dentro de la carpeta del sitio web.

ARCHIVO PRINCIPAL DE CONFIGURACIÓN DEL SERVIDOR VIRTUAL

Las siguientes directivas deberán ser incluidas en un apartado <*Directory*>dentro del fichero de configuración principal del servidor virtual. Por ejemplo, si se trata del servidor virtual por defecto, sería en ***/etc/apache2/sites-available/default***:

```
<Directory /var/www/html/profesor>
        Options Indexes FollowSymLinks
        AllowOverride None
        AuthType Basic
        AuthName "Acceso restringido"
        AuthUserFile /etc/apache2/passwd
        <RequireAll>
                Require user profesor1 profesor2
                <RequireAny>
                        Require ip 127.0.0.1
                        Require ip 192.168.37.XX
                </RequireAny>
        </RequireAll>
<Directory>
```

AuthType permite seleccionar el mecanismo de autenticación, el más común es *Basic* y está definido en *mod_auth*. El método *Basic* tiene un inconveniente y es que la contraseña se envía al servidor sin cifrar, lo que permitiría ser víctima de un ataque MIM (*man in themiddle*), por esto apache ofrece otro mecanismo llamado *Digest* el cual es mucho más seguro, está implementado en el módulo *auth_digest*. Para usarlo bastaría con cambiar la línea "**AuthType Basic"** por "**AuthTypeDigest"**.
La directiva **AuthName** permite mostrar un mensaje en el cuadro de diálogo para la contraseña.
La directiva **AuthUserFile** establece la ruta al archivo de contraseña que fue creado con *htpasswd*, donde se encuentra la información de usuarios y claves.
Finalmente, la directiva **Require** controla las IPs desde donde se podrá acceder.

FICHERO .htaccess

Permiten especificar directivas de un directorio concreto que incluso pueden sobrescribir las especificadas en el fichero de configuración del sitio. Permiten una configuración personalizada de cada directorio de *Apache* sin que intervenga el administrador que de esta forma puede delegar configuraciones. Además, no es necesario reiniciar el servidor para que los cambios tengan efecto.
Si se planea usar los archivos *.htaccess* (este nombre no es obligatorio y puede modificarse cambiando el valor en la directiva AccessFileName),se debe verificar que la configuración del servidor permite colocar directivas de autenticación en estos archivos. Para ello se necesita una directiva como la siguiente en */etc/apache2/sites-available/**default***:

AllowOverride AuthConfig (solo se sobrescriben las directivas de autenticación)
o
AllowOverride All (permite sobrescribir todas las directivas)

Cada vez que se produce una petición, el servidor busca en la ruta del recurso que ha solicitado el cliente los archivos que tengan alguno de los nombres especificados en *AccessFileName* y, aplican las directivas definidas en ellos. Estos ficheros se leen en cada petición y por tanto, los cambios establecidos se aplicarán inmediatamente.
Por defecto está configurada una directiva de sección *File* a nivel del servidor principal en el fichero *apache2.conf* (y que por tanto afecta a todos los virtuales) para que los ficheros que empiecen por *.ht* no sean accesibles ni visibles por los clientes.

<FilesMatch "^\.ht"> (**^** indica desde el principio y / escapa el **.**)
Require all denied
</FilesMatch>

El contenido que debe tener el fichero *.htaccess* podría ser el siguiente:

```
AuthType        Basic
AuthName        "Directorio protegido"
AuthUserFile    /etc/apache2/usuarios/password
Require valid-user
```

y se debe crear en el directorio del sitio web, es decir en el directorio especificado en la sección *<directory>* (en nuestro caso */var/www*). Apache buscará el archivo en ese subdirectorio y todos sus subdirectorios.

MONITORIZACIÓN

Ficheros de registros (logs) (Debian/Ubuntu)

El archivo de errores del servidor virtual se establece en **/etc/apache2/sites-available/default** con la directiva **ErrorLog**:

ErrorLog ${APACHE_LOG_DIR}/error.log

es decir:

/var/log/apache2/error.log

El archivo de accesos del servidor se establece en **/etc/apache2/sites-available/default** con la directiva **CustomLog**:

CustomLog ${APACHE_LOG_DIR}/access.log

es decir:

/var/log/apache2/access.log

Ficheros de registros (logs) (Windows)

- **Errores**
  C:\Program Files\Apache Software Foundation\Apache2.2\log/error.log

- **Accesos**
  C:\Program Files\Apache Software Foundation\Apache2.2\log/access.log

Directivas

Hay múltiples directivas para personalizar logs:

- **ErrorLog**
- **LogLevel**
- **CustomLog**

MONITORIZACIÓN

Módulos para monitorizar el servidor

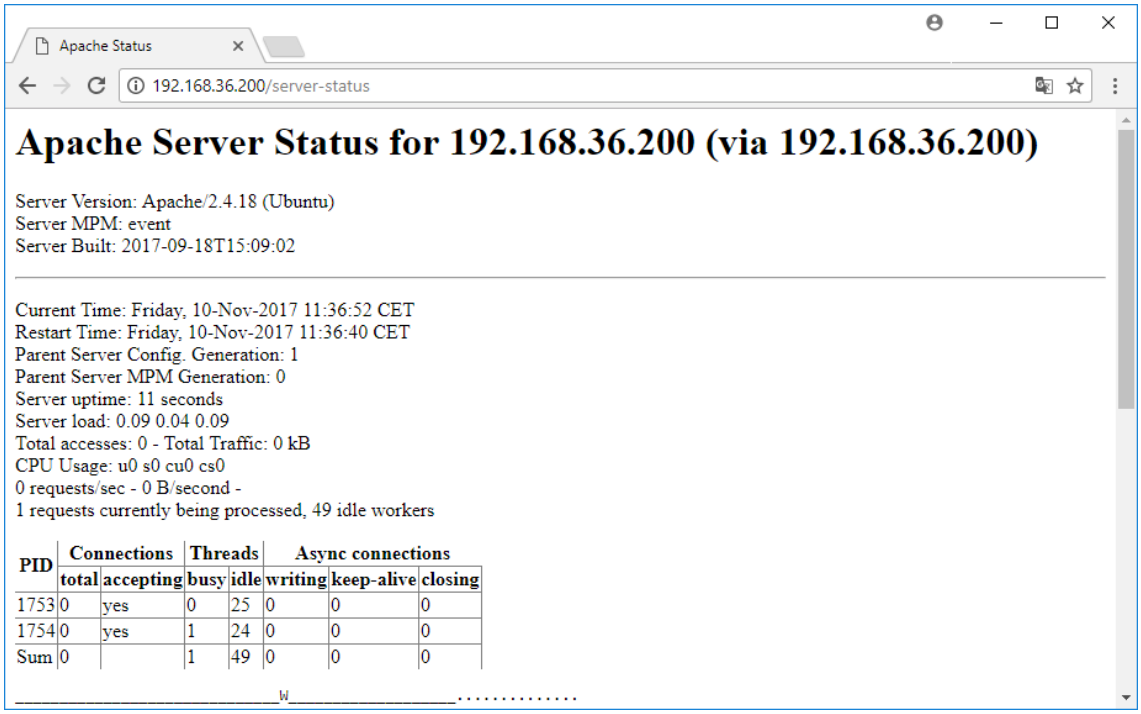
Estos dos módulos nos permiten obtener información muy útil sobre nuestro servidor.

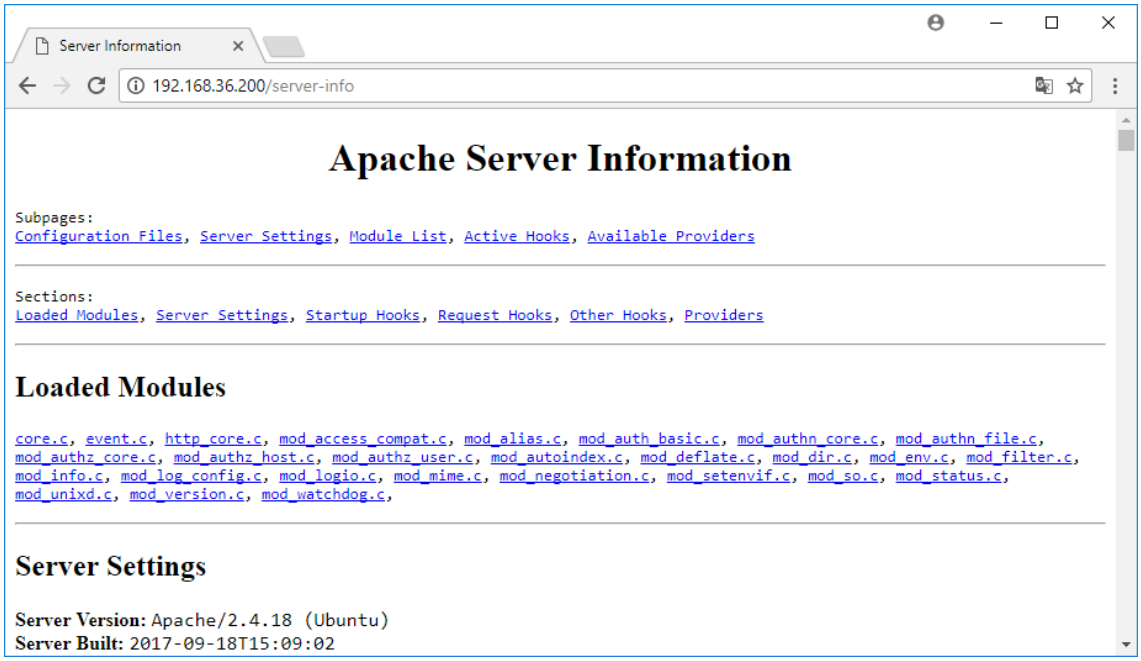
- **mod_status**. Activado por defecto.
  http://*IP_servidor*/server-status
- ***mod_info***. Hay que activar el módulo.
  sudo a2enmod info
  http://*IP_servidor*/server-info

Configuración de los módulos **mod_status:**
/etc/apache2/mods-enabled/status.conf

```
<Location /server-status>
        SetHandler server-status
        Require local
        Require ip 192.168.37.XX
        #Require ip 192.0.2.0/24
</Location>
```

Ejecución:





Analizadores de logs

- **Webalizer** (http://www.webalizer.org/)
- **Awstats** (http://www.awstats.org/)
- **Visitors** (http://www.hping.org/visitors/)
- **Analog** (http://www.analog.cx/)

Webalizer

Webalizer es un pequeño programa hecho en C que genera informes del sitio web. Gracias a esos informes, podemos observar, entre otras cosas, el número de personas que han visitado el sitio web donde se vaya a instalar(o ejecutar) el *webalizer*. Este programa no sólo nos da los informes cuantitativos, si no que también nos da informes gráficos, lo que hace mas elegante y sencillo de observar las estadísticas de nuestra página web. Entre otras cosas *webalizer* es sumamente útil para saber que archivos son los que poseen más número de descargas en nuestra página web.
Cómo ya sabemos, los accesos al sitio web, se almacenan en un fichero que se establece con la directiva ***CustomLog***:

CustomLog /var/log/apache2/access.log combined

El programa Webalizer analiza este fichero y genera unas gráficas en formato HTML para que puedan ser observadas desde el navegador. Este archivo HTML, tiene informes con gráficas de todos lo que ha sido el número de visitas, tráfico, archivos ofrecidos, el tipo de navegadores web que mas visita el sitio web, etc.

Instalación

El programa se instala con la siguiente orden:

sudo apt-get install webalizer

El contenido del fichero ***/etc/webalizer/webalizer.conf*** es similar al siguiente:

```
LogFile          /var/log/apache2/access.log
OutputDir        /var/www/html/webalizer
Incremental      no
ReportTitle     usage statics for
Hostname        ServidorLinuxXX
IgnoreSite      localhost
```

- LogFile /var/log/apache2/access.log.1". Aquí es donde especificamos el lugar donde se encuentran los informes de las personas que acceden a nuestra web, esos informes los guarda nuestro servidor web. Webalizer necesita saber dónde están esos informes para saber quienes han entrado, con que navegadores y muchas otras cosas mas.
- "OutputDir /var/www/webalizer" : Es obvio que tenemos que especificar el directorio donde se generaran los archivos HTML que contendrán las estadísticas y reportes de nuestra página web, lo más lógico es que sea en un lugar accesible desde la web.
- "Incremental no" : Sólo hay un fichero de informes. Si estuviera a "yes", habría más de un fichero. algunos servidores web (por ejemplo *apache*) parten sus bitácoras, por ejemplo, mensualmente crean un nuevo archivo de bitácoras y renombran el otro con un nombre nuevo. Si usas apache es normal que por ejemplo cada mes o cada cierto tiempo tu access.log lo renombren a access.log.1 y se cree un nuevo access.log vacío donde se guardarán los próximos nuevos registros. Está opción establece que *Webalizer* va a seguir tomando en cuenta los archivos ya renombrados, como también los nuevos archivos.
- "ReportTitle usage statics for" :Se trata del título que quieres que lleve las páginas HTML que se generarán.
- "HostName URL servidorLinuxXX" : Aquí se coloca el Nombre o URL del servidor donde está el *webalizer*. Este Nombre normalmente aparece al lado del título de Reportes (ReportTitle).
- "IgnoreSite localhost" : Esto lo que hace es ignorar o no tomar en cuenta los registros que vengan de la computadora local. Si no colocamos está opción cada vez que nosotros mismos desde la máquina en que está el servidor web visitáramos la página se registraría ese acceso y muchas veces no queremos esto. ¿Por qué? Porque normalmente hacemos prácticas de diseño o modificación de la página web lo que ocasiona que estamos entrando a la página web y si obviamos esta opción estaríamos haciendo reportes o estadísticas de cuantas veces nosotros mismos estamos entramos a la página (y no queremos eso).

Configuración

- Crear el directorio ***/var/www/html/webalizer***.
- Modificar el fichero ***/etc/webalizer/webalizer.conf*** para quitar la extensión .1 al fichero de informes y especificar **/var/www/html/webalizer** como directorio de salida:

```
LogFile /var/log/apache2/access.log

# LogType defines the log type being processed.  Normally, the Webalizer
# expects a CLF or Combined web server log as input.  Using this option,
# you can process ftp logs (xferlog as produced by wu-ftp and others),
# Squid native logs or W3C extended format web logs. Values can be 'clf',
# 'ftp', 'squid' or 'w3c'.  The default is 'clf'.

#LogType        clf

# OutputDir is where you want to put the output files.  This should
# should be a full path name, however relative ones might work as well.
# If no output directory is specified, the current directory will be used.

OutputDir /var/www/html/webalizer
```

Generar informes

Para generar el documento *html* (se almacena en la carpeta **webalizer**) con las estadísticas se ejecuta la siguiente orden:

sudo webalizer

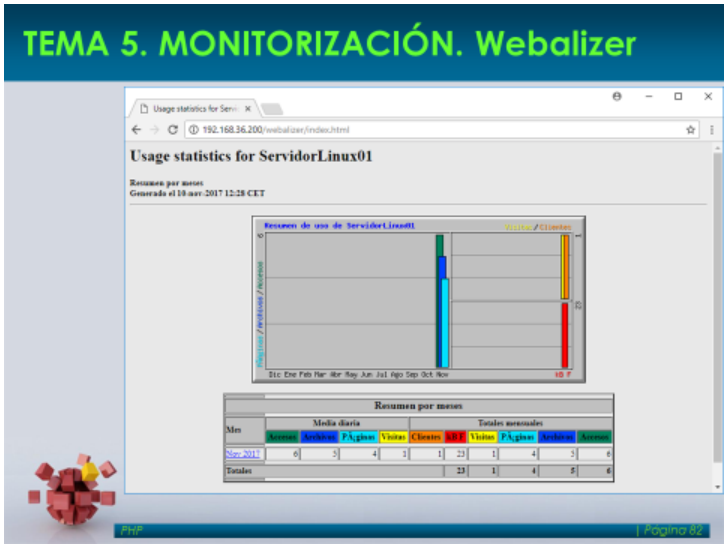
Hay que tener en cuenta que *webalizer* sólo genera estos informes las veces que ejecutemos el programa. Por lo tanto tendríamos que estar ejecutando el programa cada vez que queramos generar o actualizar las estadísticas ya existentes. *¿Resulta esto muy tedioso no?* .En Linux y en los todos los sistemas derivados de Unix existe una herramienta llamada **cron**, que lo que hace es ejecutar tareas en intervalos de tiempo especificados por el usuario. Para poder utilizar el *cron* hay que editar su archivo de configuración ***(/etc/crontab***) y agregar la siguiente línea:

0 0 * * * /usr/bin/webalizer -c /etc/webalizer/webalizer.conf -d

Aquí lo que le estamos diciendo es que ejecute ese comando todos los días a las 0 horas (media noche).

Visualizar estadísticas

Desde el navegador:



ALOJAMIENTO VIRTUAL DE SITIOS WEB

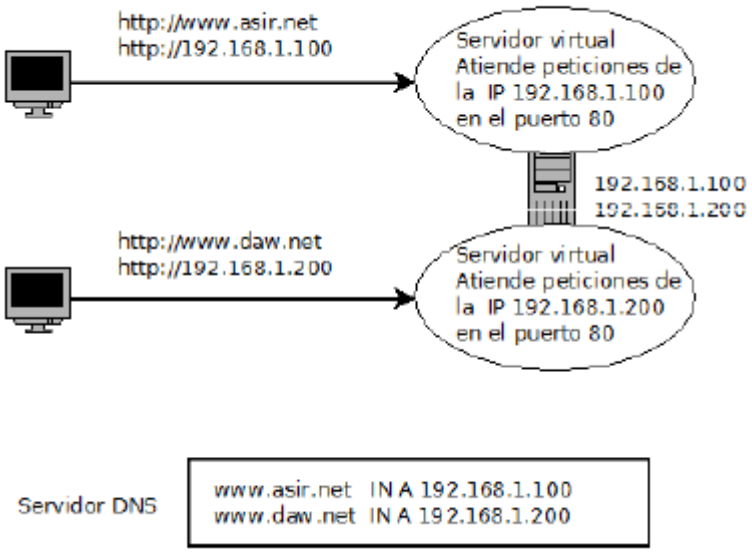
El alojamiento virtual de sitios web (*web virtual hosting*) consiste en simular que existen varias maquinas (hosts) con sus respectivos sitios web sobre un solo. Por ejemplo **www.asir.net** y **www.daw.net** en un mismo servidor.

Objetivos:

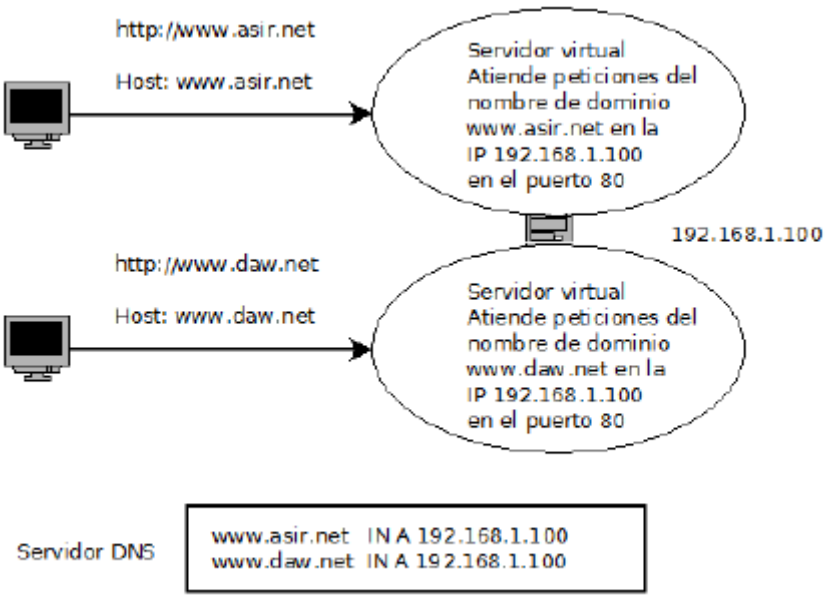
- Reducción del número de máquinas físicas.
- Aprovechar mejor los recursos (uso de CPU, memoria,...) de los equipos.
- Aprovechar mejor las direcciones IP disponibles.

Existen tres formas diferentes de crear servidores virtuales en Apache:

1. Basados en **IP**: En este método se necesita configurar las direcciones IP de cada sitio en Apache. El servidor físico tendrá varias direcciones IP, una para cada servidor web virtual. Cada servidor virtual atenderá peticiones en una dirección IP diferente. A efectos de los usuarios es como si existiesen varios servidores web, uno en cada dirección IP.
   Para realizar esta configuración la máquina donde se ejecuta el servidor web debe tener varias tarjetas de red configuradas cada una con una dirección IP o hay que asignar varias direcciones IP (alias o interfaces virtuales) a una misma tarjeta de red (la mayoría de los sistemas operativos actuales soportan esta funcionalidad).



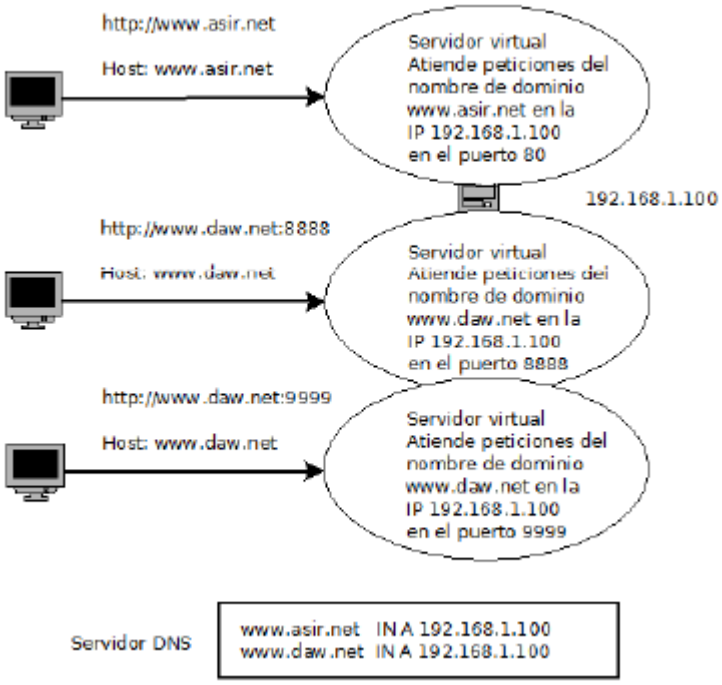
2. Basados en **nombres**: Este tipo es la opción más común. Se configura todo para que múltiples dominios DNS apunten a una única máquina con Apache (por ejemplo *www.daw1.es* y *www.daw2.es*). Cada servidor virtual atiende las peticiones de un nombre de dominio. Requiere configuración del servidor DNS para que funcione. Es el **único método que vamos a estudiar**.



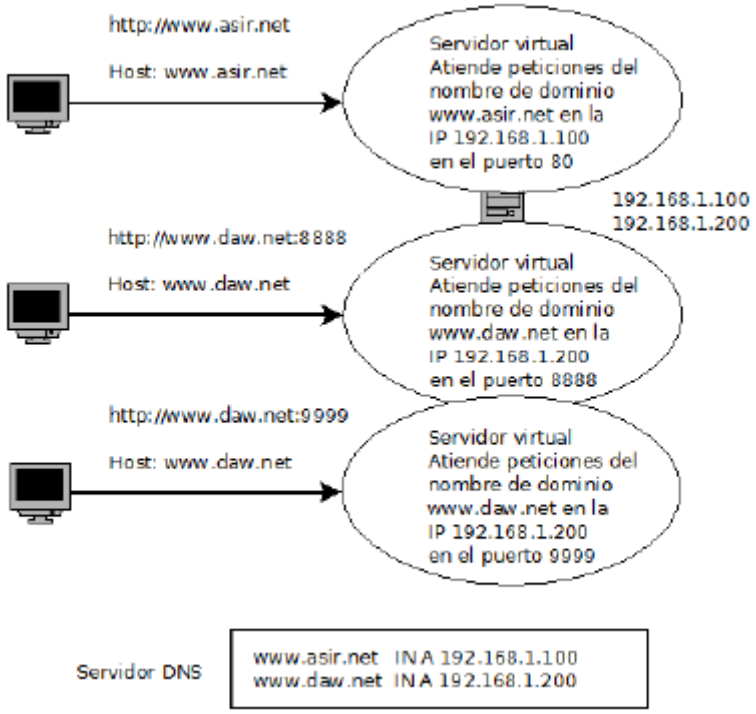
Permiten alojar múltiples dominios en una única máquina. Ventajas

- Ahorro de equipos.
- Ahorro de IPs.
- Facilita la administración centralizada.

3. Basados en **puertos**. Cada sitio se atenderá en la misma IP o nombre pero en distintos puertos. Es una extensión de cualquiera de las alternativas anteriores.



4. Combinaciones. En un mismo servidor web se pueden combinar servidores virtuales basados en IP, en nombres y en puertos.



ALOJAMIENTO VIRTUAL EN APACHE

Apache admite configuraciones de servidores virtuales basados en IPs, basados en nombres y basados en puertos. Estudiaremos los servidores virtuales basados en nombres.

Aunque cada *virtualhost* es único e independiente de los demás, todo aquello que no esté incluido en la definición de cada *virtualhost* se heredará de la configuración principal: **apache2.conf** (*/etc/apache2/apache2.conf*), así, si quieres definir una directiva común en todos los *virtualhost* no debes modificar cada uno de los *virtualhost* introduciendo esa directiva sino que debes definir esa directiva en la configuración principal del servidor web Apache, de tal forma que todos los *virtualhost* heredarán esa directiva, por ejemplo en apache2.conf puedes encontrar la directiva *Timeout 300*, que establece la directiva *Timeout* igual a 300 segundos, esto es, indica el número de segundos antes de que se cancele un conexión por falta de respuesta.

Configuración (DEBIAN/UBUNTU)

Apache2 trae una configuración predeterminada preparada para servidores virtuales. Viene configurado con un único servidor virtual predeterminado (usando la directiva **VirtualHost**) que funcionará como servidor predeterminado gracias al fichero */etc/apache2/sites-enabled/**000-default*** que realmente es una enlace que apunta al *fichero /etc/apache2/sites-available/**default***.

Nota: En el fichero /etc/apache2/apache2.conf está incluida la directiva **Include sites-enabled** que provoca que se apliquen las configuraciones establecidas en los ficheros que se almacenan en este directorio.

```
alumno@ServidorLinux00:~$ ls /etc/apache2/sites-available
000-default.conf  default-ssl.conf
```

```
alumno@ServidorLinux00:~$ ls -l /etc/apache2/sites-enabled
total 0
lrwxrwxrwx 1 root root 35 nov  8 12:51 000-default.conf -> ../sites-available/000-default.conf
```

Podemos consultar la configuración del servidor virtual por defecto:

sudo nano /etc/apache2/sites-available/000-default.conf

Y nos mostrará un contenido similar a este:

```
<VirtualHost *:80>
        # The ServerName directive sets the request scheme, hostname and port that
        # the server uses to identify itself. This is used when creating
        # redirection URLs. In the context of virtual hosts, the ServerName
        # specifies what hostname must appear in the request's Host: header to
        # match this virtual host. For the default virtual host (this file) this
        # value is not decisive as it is used as a last resort host regardless.
        # However, you must set it for any further virtual host explicitly.
        #ServerName www.example.com

        ServerAdmin webmaster@localhost
        DocumentRoot /var/www/html

        # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
        # error, crit, alert, emerg.
        # It is also possible to configure the loglevel for particular
        # modules, e.g.
        #LogLevel info ssl:warn

        ErrorLog ${APACHE_LOG_DIR}/error.log
        CustomLog ${APACHE_LOG_DIR}/access.log combined

        # For most configuration files from conf-available/, which are
        # enabled or disabled at a global level, it is possible to
        # include a line for only one particular virtual host. For example the
        # following line enables the CGI configuration for this host only
        # after it has been globally disabled with "a2disconf".
        #Include conf-available/serve-cgi-bin.conf
</VirtualHost>
```

El fichero comienza con la definición del virtual host:

<VirtualHost *:80>

básicamente, lo que se dice es que este sitio web podrá ser accedido a través de cualquiera de las interfaces de red (tarjetas de red) del servidor por el puerto 80.

En el archivo podemos observar que el directorio raíz es ***/var/www/html*** y la configuración que tiene el directorio. Ten en cuenta que es necesario reiniciar Apache para que los cambios en la configuración surtan efecto.
Un directorio hereda las directivas del superior si no se sobrescriben. Por ello, todos los directorios que creemos dentro de ***/var/www/html*** tendrán la misma configuración que el raíz excepto en aquellas directivas que especifiquemos en la configuración propia del directorio.

DIRECTORIOS

- **/etc/apache2/sites-available/**
  - Directorio que contiene los ficheros de configuración de servidores virtuales disponibles.
  - Por defecto están creados el fichero default con la configuración del servidor virtual por defecto y el fichero default-ssl con la configuración de un servidor por defecto seguro.

```
alumno@ServidorLinux01:/etc/apache2/sites-available$ ls
default  default-ssl
```

- **/etc/apache2/sites-enabled/**
  - Directorio que contiene enlaces simbólicos a los ficheros de **sites-enabled**.
  - Los enlaces serán los servidores virtuales habilitados.
  - Se incluyen en apache2.conf en orden alfabético.
  - Por defecto está creado el fichero **000-default.conf** que es un enlace al fichero *000-default.conf* del directorio **sites-available**.

```
alumno@ServidorLinux00:~$ ls -l /etc/apache2/sites-enabled
total 0
lrwxrwxrwx 1 root root 35 nov  8 12:51 000-default.conf -> ../sites-available/000-default.conf
```

En el directorio **/etc/apache2/sites-available** guardaremos todos los archivos que tienen la configuración de los *VirtualHost* definidos en Apache. En **/etc/apache2/sites-enabled/** están los enlaces a las configuraciones definidas en el directorio anterior, indicando cuáles de ellas están activas. De esta forma, tenemos una manera rápida y cómoda de gestionar nuestros hosts virtuales.

PASOS A SEGUIR PARA HABILITAR UN SERVIDOR VIRTUAL

1. Consultar el fichero **/etc/apache2/ports.conf** y comprobar que están habilitados los servidores virtuales por nombre en todas las direcciones IP y en el puerto 80 (directiva **NameVirtualHost *:80**).

2. Activar el módulo de virtual hosts (**vhost_alias**) no hace falta instalar ningún paquete de software adicional, por lo tanto sólo hay que hacer lo siguiente:

   sudo a2enmod vhost_alias

3. Luego es necesario reiniciar apache, para lo cual usaremos el siguiente comando:

   sudo /etc/init.d/apache2 restart

4. Si queréis verificar que el módulo efectivamente ha sido activado basta con echar un vistazo en la carpeta **/etc/apache2/mods-enabled**y veréis que se ha creado un enlace simbólico nuevo llamado **vhost_alias.load**.

   ls–l/etc/apache2/mods-enabled

5. Crear un directorio en donde se almacenarán los ficheros del sitio web:

   sudo mkdir/usr/local/apache2/ejemplo

6. Crear el fichero de configuración del sitio web. Este fichero debe ser creado dentro de la carpeta **/etc/apache2/sites-available**. El fichero puede tener cualquier nombre, generalmente alguno relacionado con el dominio del sitio web(*ejemplo.ejm*), para nuestro caso de ejemplo le llamaremos **ejemplo**.

   sudo nano /etc/apache2/sites-available/ejemplo.conf

   El contenido podría ser el siguiente:

```
<VirtualHost *:80>
        ServerName ejemplo.ejm
        DocumentRoot  /var/www/html/ejemplo
        ErrorLog ${APACHE_LOG_DIR}/ejemplo_error.log
        CustomLog ${APACHE_LOG_DIR}/ejemplo_access.log combined
        <Directory /var/www/html/ejemplo>
                Options –Indexes
                AllowOverride None
                Require all granted
        </Directory>
</VirtualHost>
```

Ojo, no se pone *FollowSymLinks* con -Indexes

8. Activar el nuevo virtual host:

   sudo a2ensite ejemplo.conf

   Se creará el enlace *ejemplo.conf* en el directorio ***/etc/apache2/sites-enabled*** que apunta al mismo fichero en ***/etc/apache2/sites-available***. Los enlaces indican qué hosts están activos. Puedes verificarlo con el comando:

   ls –l /etc/apache2/sites-enabled

   Para desactivar un host hay que usar la orden **a2dissite.**

9. Reiniciar Apache:

   sudo /etc/init.d/apache2 restart

Una vez ejecutados estos pasos, podemos crear nuestro ficheros en el directorio del nuevo sitio web. Inicialmente basta con crear un sencillo fichero *index.html* y después podemos acceder al sitio web desde el navegador para visualizarlo:

http://ejemplo.ejm

Importante: un servidor dns debe poder resolver el nombre de dominio **ejemplo.ejm**.

Modificando los mensajes de error

La directiva ***ErrorDocument*** sirve para establecer mensajes de error personalizados para diferentes situaciones. Los códigos de error que pueden utilizarse son los definidos por la organización w3 para el protocolo http en este caso la versión 1.1 y se pueden consultar aquí. En esta otra página seexplican de una manera más comprensible.
Esta directiva se usará tantas veces como se crea necesario. Se establece de manera independiente para cada sitio virtual y se suele escribir debajo de *CustomLog*.
Hay una variable de entorno llamada *APACHE_LOG_DIR que se usa* para determinar el directorio donde se ubicarán los archivos de log. En esta instalación es */var/log/apache2/* pero si no lo sabemos, podemos buscar los archivos en la estructura de directorios o consultar el valor de la variable de entorno.
Aquí tenemos ejemplos de utilización de esta directiva:

```
ErrorDocument 404 /cgi-bin/bad_urls.pl
ErrorDocument 401 /subscription_info.html
ErrorDocument 403 "Disculpe, es imposible acceder a la página"
```

Alias a otros directorios

Por defecto la estructura de un sitio web será la que demos a los directorios desde el punto de montaje del directorio raíz del sitio, sin embargo es posible incluir otros directorios del árbol de nuestro servidor y hacer que parezcan parte del sitio como cualquier otra carpeta que esté contenida físicamente dentro. Es algo similar a tener un enlace simbólico a archivos o carpetas en otro directorio.
En el archivo de configuración del sitio principal visto antes ya existe un alias a un directorio que se encuentra fuera de la carpeta definida para el sitio.
Para realizar esta operación se utiliza la directiva ***Alias*** y justo debajo (no es obligatorio pero sí una buena práctica) se configura con la directiva *Directory* como cualquier otro directorio.

/etc/apache2/sites-available/000-default.conf

```
Alias /apuntes /home/alumno/apuntes
<Directory /home/alumno/apuntes/>
        Options Indexes FollowSymLinks
        AllowOverride None
        Require all granted
</Directory>
```

Redirecciones

Algunas veces es útil poder redirigir las llamadas a una dirección web para que se procesen en otro punto. Por ejemplo si hemos cambiado la estructura de nuestro sitio y hemos reubicado la página de contacto podemos querer que los usuarios que accedan mediante la antigua dirección sean redirigidos automáticamente a la nueva. También puede ser útil si hemos dividido nuestro sitio para facilitar la gestión o el mantenimiento.
Para ello se usa la directiva ***Redirect*** que nos permite asociar una dirección absoluta o relativa a otra.

/etc/apache2/sites-available/000-default.conf

```
        Redirect /fp http://www.totofp.es
</VirtualHost>
```

PROTOCOLO HTTPS. ACCESO A PÁGINAS SEGURAS (SSL/TLS)

El tráfico de red generado entre el cliente (navegador) y el servidor Web apache en el puerto 80 no está encriptado, viaja tal cual por la red (sin cifrar). Entonces es posible, con la herramienta adecuada interceptar y observar este tráfico y obtener entre otras cosas contraseñas, números de tarjetas de crédito, etc. Por ejemplo, imaginemos que estamos rellenando un formulario donde introducimos nuestro nombre de usuario y contraseña. Cuando enviamos el formulario, esos datos viajarán por la red tal cual los hemos introducido y si en algún punto la comunicación es interceptada, el ladrón de los datos podrá conocerlos sin ningún problema. Este inconveniente es más notable si pensamos en datos bancarios, números de tarjetas de crédito, etc.

Por lo tanto se hace necesario una manera de proteger la información sensible que enviamos o recibimos, de tal manera que si es interceptada, no pueda ser conocida por terceras personas. Para eso se usa el protocolo **https**.

Una página web segura o un sitio web seguro es un sitio web que utiliza el protocolo ***https*** en lugar de utilizar el protocolo *http*. ***https*** es una implementación de HTTP sobre el protocolo SSL o TSL en el servidor.

El protocolo ***https*** es idéntico al protocolo http con la excepción de que la transferencia de información entre el cliente (navegador web) y el servidor (servidor web) viaja a través de Internet cifrada utilizando robustos algoritmos de cifrado de datos y aunque es posible interceptarlo y observarlo, no se verá más que basura o cadenas de caracteres sin ningún significado todo el tiempo, logrando así un canal seguro cifrado entre el cliente y el servidor.

https es un poco menos eficiente que HTTP por lo que si se utiliza en sitios donde la transferencia de información es muy grande puede notarse en el rendimiento. Por supuesto en el caso de que la información trasferida sea sensible, las ventajas compensan con creces a los inconvenientes.

Existen dos modos: Uno en el que solo el servidor demuestra su identidad y otro en el que tanto el cliente como el servidor usan Certificados Digitales.

Este protocolo utiliza un cifrado basado en *Secure Socket Layers* (SSL, Capa de Conexiones Seguras), recientemente sustituido por *Transport Layer Security* (TLS) basado en SSL y ambos son compatibles. De manera predeterminada se usará el puerto 443 para las conexiones.

Certificado Digital SSL/TLS

Un certificado digital es un archivo de aproximadamente 1K otorgado por una autoridad de certificación que garantiza la asociación de una persona física o entidad con una firma digital, es decir, que es quien dice ser.

Es un documento digital único que garantiza la vinculación entre una persona o entidad con su clave pública.

Contiene información de su propietario como nombre, dirección, correo electrónico, organización a la que pertenece, su clave pública, periodo de validez, firma digital, nombre de la AC que emitió, firma digital de la AC …

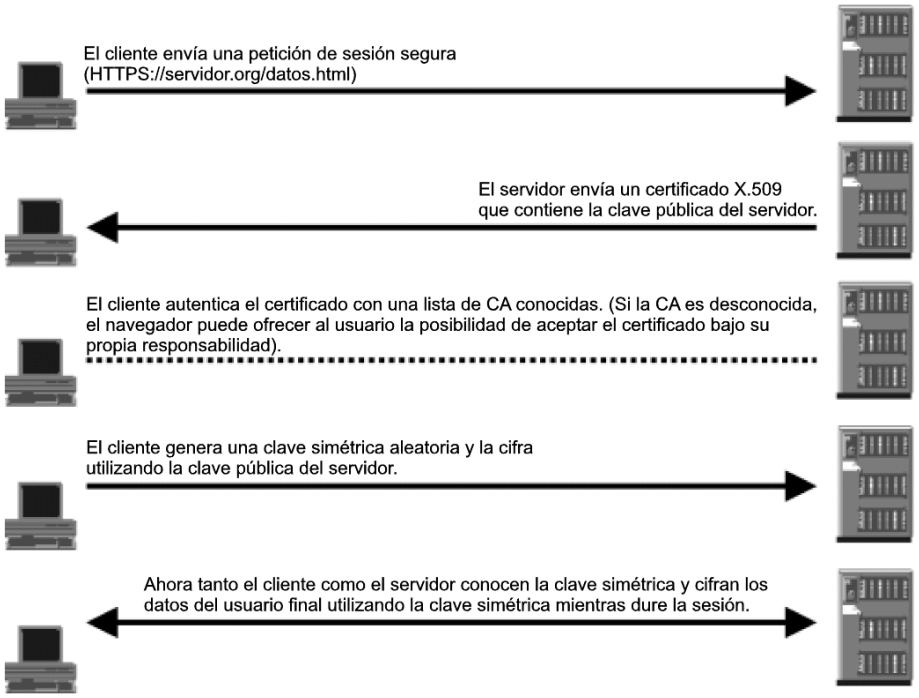
Cuando se solicita el certificado a la autoridad certificadora (AC) se le pasan a esta los datos mencionados y la AC tomando como base estos datos genera un número primo de gran tamaño al que aplica una función resumen como SHA (SHA1, SHA2… en función de la longitud de las claves obtenidas) y genera la huella digital que es una cadena de 160 bits que visualiza como una cadena de 20 números en hexadecimal. A esta huella la AC le aplica un algoritmo de clave asimétrica como RSA y obtiene la firma del certificado.

Cuando un cliente se conecta a un servidor web, este envía el certificado para demostrar que es quien dice que es. Sólo se envía cifrada la firma del certificado, el resto del certificado se envía sin cifrar.

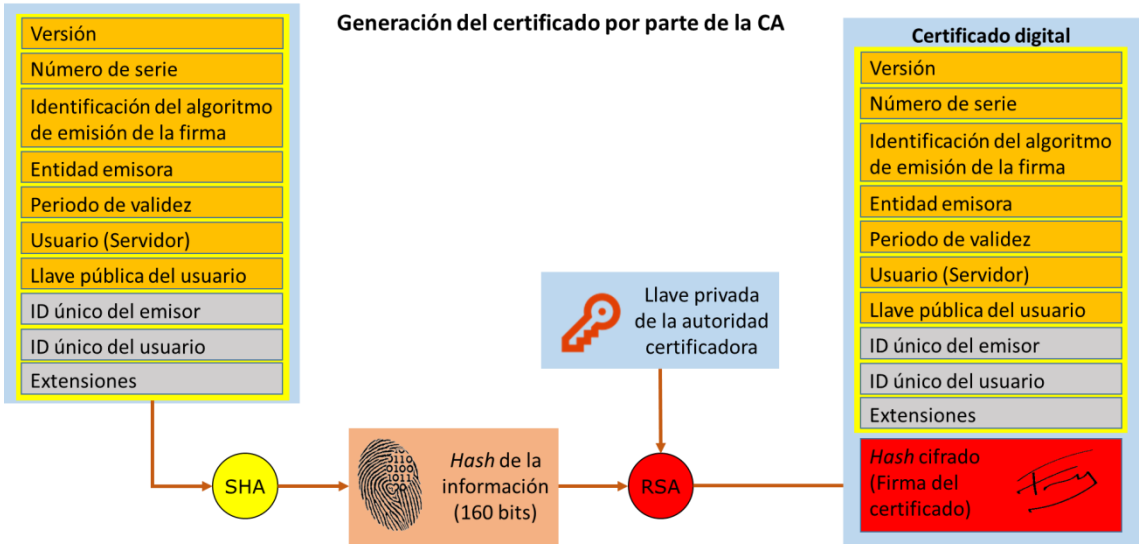
El cliente por su parte, partiendo de la información del certificado, vuelve a generar con SHA (como ya hizo la AC) la huella digital y, por otra parte, utilizando la clave pública de la AC (esta clave se almacena en el navegador) y un algoritmo como RSA sobre la firma cifrada del certificado genera también la huella digital del certificado. Si ambas huellas digitales generadas de formas diferentes coinciden, el certificado es auténtico.

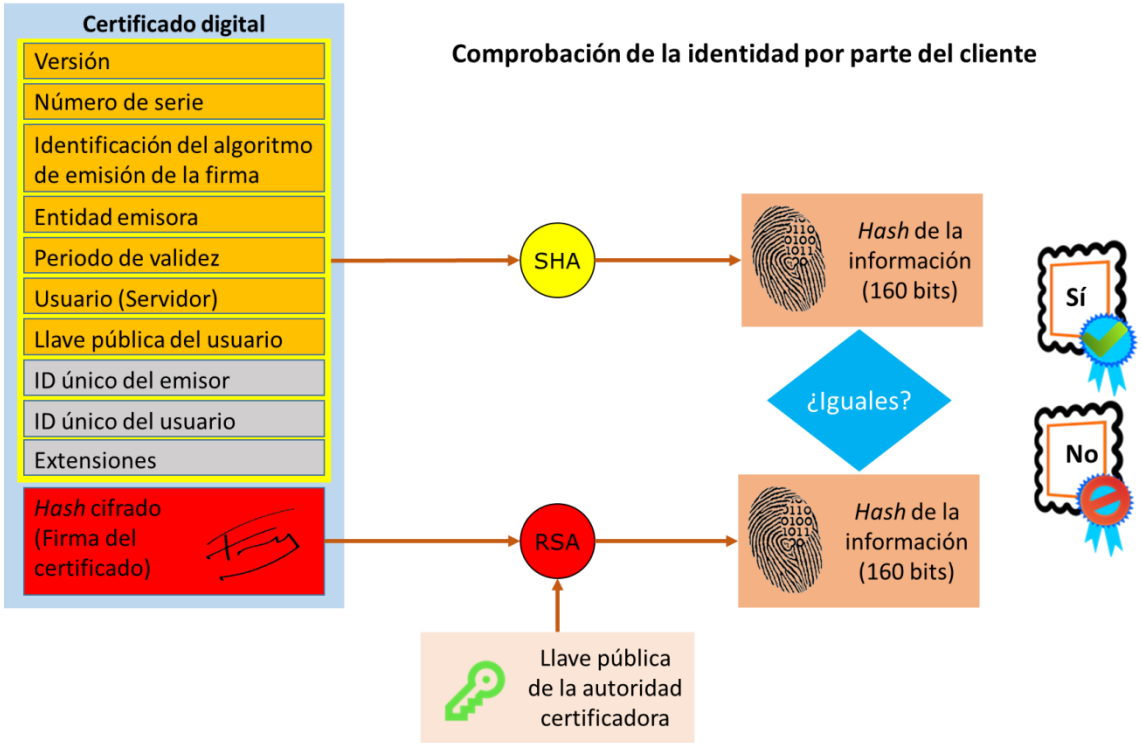
Finalmente, el cliente envía al servidor la clave simétrica (es decir, ya sólo se utilizará una clave y no dos como antes) que se utilizará a partir de ahora en la comunicación. Esta clave se envía cifrada con la clave pública y el servidor la descifra con su clave privada. A partir de este momento, se establece un canal seguro donde toda la información viaja cifrada utilizando una clave simétrica (es decir, una única clave) y un algoritmo como RSA o RC4.

Lo primero que debemos tener instalado en nuestro ordenador para utilizar HTTPS son los certificados raíz de las autoridades certificadoras (este certificado consta la clave pública de la Autoridad de Certificación y por tanto será necesario para comprobar la autenticidad de cualquier certificado emitido por ella.
Cuando usamos un navegador web determinado, la empresa que lo ha desarrollado ya ha introducido en él determinadas Autoridades Certificadoras que considera de confianza. El usuario confía en los desarrolladores del navegador para que se aseguren de que esos certificados son de confianza. Si el certificado que nos envía una Web no está firmado por ninguna de las AC's que tenemos guardadas en nuestro navegador, nos preguntará si queremos aceptarlo bajo nuestra responsabilidad. Para añadir una nueva CA solo tendremos que descargar el archivo correspondiente (extensión ***.cer***) y proceder a instalarlo en nuestro navegador habitual.



Para poder utilizar HTTPS en nuestro servidor es necesario disponer de un Certificado Digital. Podemos obtener un Certificado Digital de una Autoridad Certificadora o crear nuestra propia Autoridad Certificadora y generar nuestros Certificados Digitales.
En España, podemos obtener Certificados Digitales a través de la Fábrica Nacional de Moneda y Timbre o mediante empresas como *Verisign*.





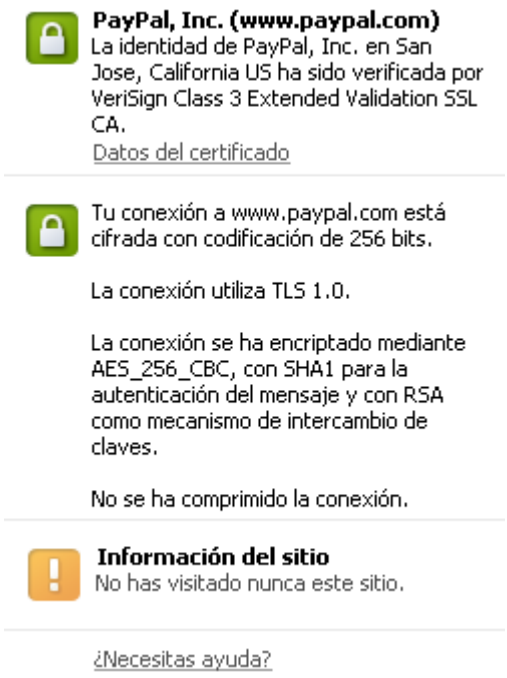
Navegando por Webs seguras

Cuando la conexión que hemos establecido con una Web es segura, el navegador lo indica, normalmente, con un candado cerrado. Pulsando sobre dicho icono tendremos acceso a todos los datos del certificado y podremos ver quién es el propietario y cuál es la Autoridad Certificadora del mismo. También podremos ver datos como la fecha de caducidad, el algoritmo de cifrado, la clave pública, etc.

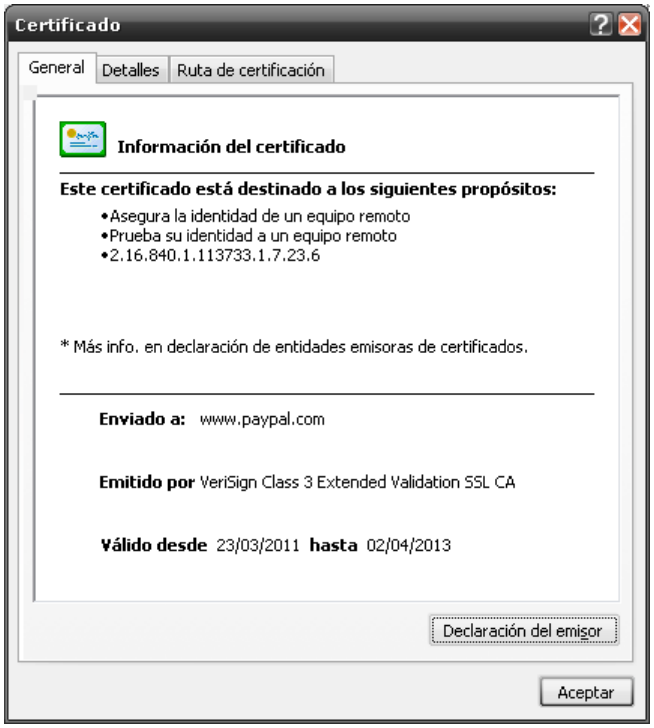
Ejemplo de Web segura con certificado de una CA conocida

Cuando entramos en *www.paypal.com* comprobamos que usa una conexión segura. Al no obtener ningún mensaje de aceptación del certificado, significa que está firmado por una CA que ya tenemos instalado en nuestro sistema.

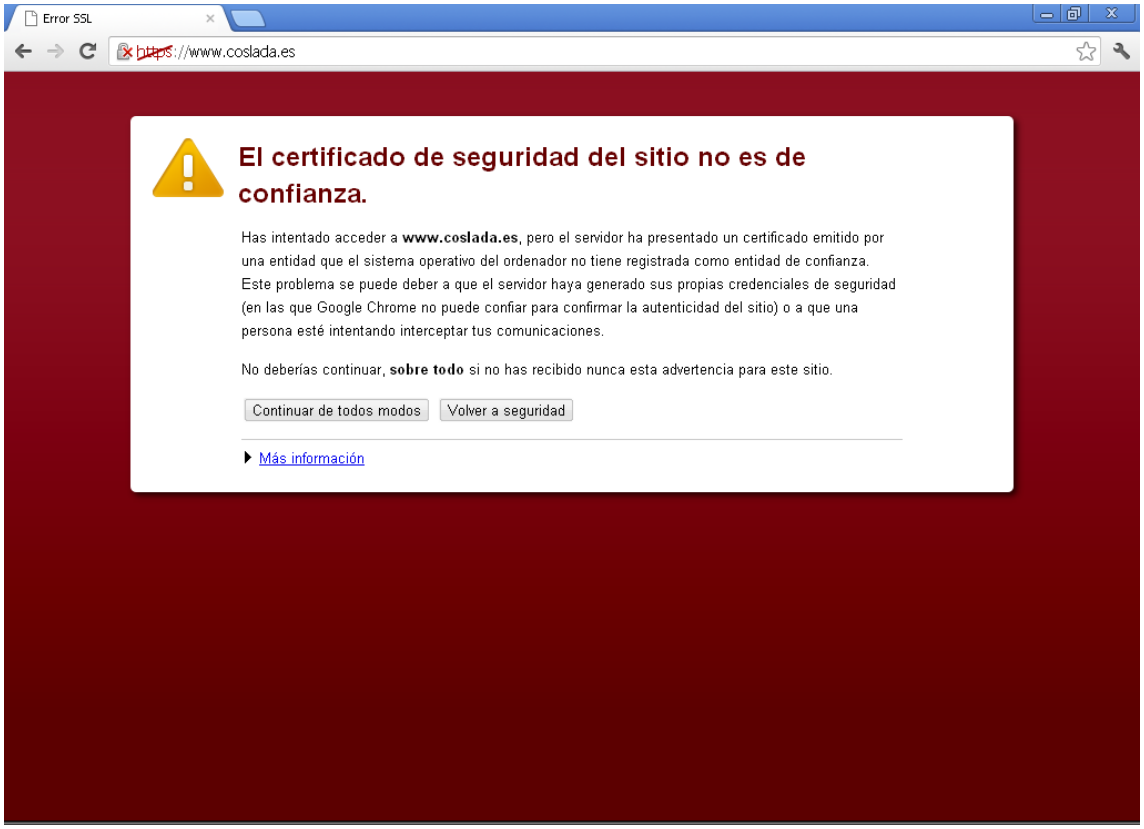
Pulsando sobre el icono del candado podemos comprobarlo.



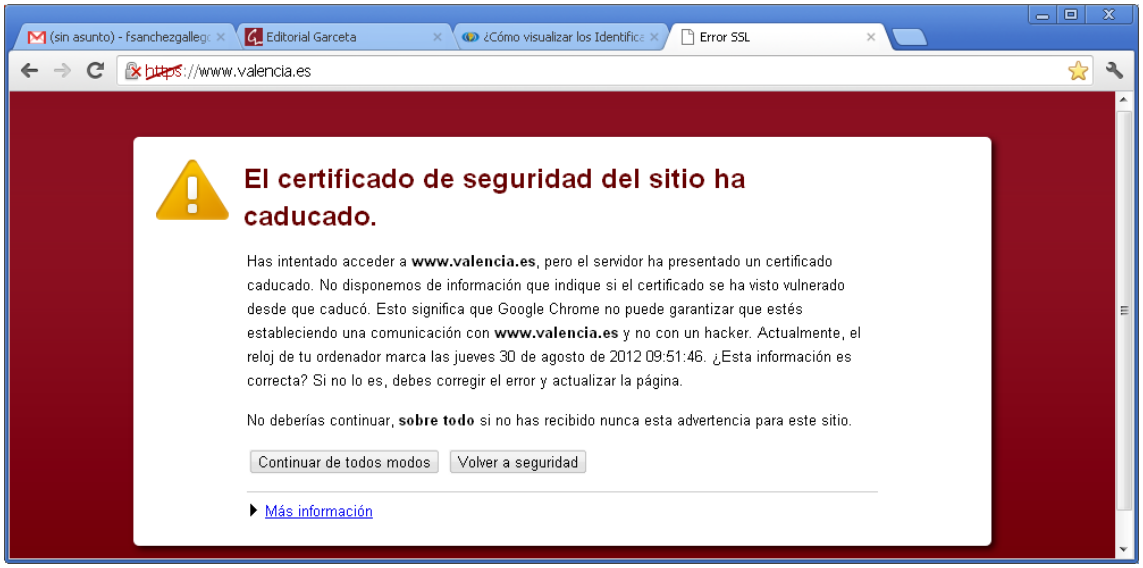
Al abrir la información del certificado, podemos observar los datos del certificado como la Autoridad Certificadora emisora, el destinatario del mismo y periodo de validez del certificado.



Ejemplo de Web segura con certificado de una CA desconocida



Ejemplo de Web segura con certificado de una CA caducado



Certificados no otorgados por una autoridad de certificación

Un servidor web, como Apache, puede emitir certificados, pero puede que en algún navegador sea interpretado como peligroso, esto suele ser debido a que los navegadores poseen en su configuración una lista de Entidades Certificadoras que verifican, autentican y dan validez a los certificados. Eso no quiere decir que no puedes crear tus certificados en un servidor web, de hecho muchas empresas lo hacen, sobre todo para sitios internos o externos en los que solamente puede acceder personal autorizado por la propia empresa. Ahora si, si utilizas certificados mediante Apache en un sitio visible a través de Internet y accesible por cualquier usuario, lo más probable es que el usuario no aceptará la comunicación, por visionar en el navegador un aviso de problema de seguridad.

Configuración de Apache + SSL/TSL en Linux Ubuntu

La configuración de Apache + SSL/TLS en Ubuntu es muy sencilla, la distribución provee de un paquete que permite la generación e instalación de los certificados digitales necesarios. Es importante resaltar que este certificado es auto-firmado y como tal, no está avalado por ninguna autoridad de certificación y, por lo tanto, siempre aparecerá una advertencia en la pantalla del navegador indicando que no se puede verificar el origen del mismo. Pero por ahora, como herramienta de prueba y que permita una primera configuración rápida es suficiente. Los certificados digitales generados por autoridades de certificación como Verisign, Thawte, etc., suelen costar varios cientos de euros.
Apache viene configurado con un servidor virtual por defecto que hay que habilitar:

sudo a2ensite default-ssl

No hay que olvidar reiniciar el servidor.
Observa el contenido del fichero **/etc/apache2/sites-availables/default-ssl.conf** y observa su configuración. Fíjate en las directivas que habilitan SSL y que definen la ruta del certificado digital que usará el servidor:

```
<IfModule mod_ssl.c>
        <VirtualHost _default_:443>
                ServerAdmin webmaster@localhost

                DocumentRoot /var/www/html

                # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
                # error, crit, alert, emerg.
                # It is also possible to configure the loglevel for particular
                # modules, e.g.
                #LogLevel info ssl:warn

                ErrorLog ${APACHE_LOG_DIR}/error.log
                CustomLog ${APACHE_LOG_DIR}/access.log combined
```

```
SSLEngine on

#   A self-signed (snakeoil) certificate can be created by installing
#   the ssl-cert package. See
#   /usr/share/doc/apache2/README.Debian.gz for more info.
#   If both key and certificate are stored in the same file, only the
#   SSLCertificateFile directive is needed.
SSLCertificateFile      /etc/ssl/certs/ssl-cert-snakeoil.pem
SSLCertificateKeyFile /etc/ssl/private/ssl-cert-snakeoil.key
```

donde,

SSLEngine on : Activa o desactiva SSL

SSLCertificateFile /etc/ssl/certs/ssl-cert-snakeoil.pem: Certificado digital del propio servidor Apache

SSLCertificateKeyFile /etc/ssl/private/ssl-cert-snakeoil.key: Clave privada del servidor Apache.

Si quisiéramos habilitar un servidor virtual *https* para el dominio ***seguro.dawXX.net***. Pasos a seguir:

1. Instala los paquetes necesarios para la generación del certificado:

   sudo apt-get install openssl

2. Crea el directorio **/var/www/html/seguro** como directorio raíz del sitio.
3. Crea el fichero **/var/www/html/seguro/index.html** con el contenido que quieras.
4. Sitúate en el directorio *home* del usuario con el que has iniciado sesión.
5. Crea una clave privada RSA de 2048 bit:

   sudo openssl genrsa –out seguro.key 2048

```
alumno@ServidorLinux01:~$ openssl genrsa -out seguro.key 2048
Generating RSA private key, 2048 bit long modulus
.............................+++
.....................................................................
.................................................+++
e is 65537 (0x10001)
alumno@ServidorLinux01:~$
```

6. Genera una solicitud de certificado:
   sudo openssl req –new –key seguro.key -out seguro.csr

```
alumno@ServidorLinux01:~$ openssl req -new -key seguro.key -out seguro.csr
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:ES
State or Province Name (full name) [Some-State]:Madrid
Locality Name (eg, city) []:Madrid
Organization Name (eg, company) [Internet Widgits Pty Ltd]:daw01
Organizational Unit Name (eg, section) []:daw01
Common Name (e.g. server FQDN or YOUR name) []:seguro.daw01.net
Email Address []:admin@daw01.net

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:
alumno@ServidorLinux01:~$ _
```

Este comando pedirá algunos datos: nombre de empresa, país, contraseña, etc. Y genera dos archivos:

- La clave privada: *seguro.key*.
- El certificado con la clave pública de la AC: *seguro.csr*

Este comando pedirá algunos datos: nombre de empresa, país, contraseña...

7. Genera un certificado. Hay dos opciones:
   A) Enviar la solicitud de certificado a una autoridad de certificación. Se envía, habitualmente, a través de un formulario web a la empresa certificadora que:
      - Validan si se ha pagado (si es necesario).
      - Validan la solicitud.
      - Generan el Certificado (.crt).
      - Remiten el certificado e instrucciones.
   B) Crear el certificado digital autofirmado usando la clave privada:

sudo openssl x509 –req –sha256 –days 365 –in seguro.csr –signkey seguro.key –out seguro.crt

```
alumno@ServidorLinux01:~$ openssl x509 -req -days 365 -in seguro.csr -signkey se
guro.key -out seguro.crt
Signature ok
subject=/C=ES/ST=Madrid/L=Madrid/O=daw01/OU=daw01/CN=seguro.daw01.net/emailAddre
ss=admin@daw01.net
Getting Private key
alumno@ServidorLinux01:~$ _
```

La opción -**days 365** significa que el certificado tardará 1 año en caducar.

8. Copia la clave y el certificado en los directorios que utiliza por defecto Apache y configura los permisos adecuados:

   - **sudo mv seguro.key /etc/ssl/private/**
   - **sudo mv seguro.crt /etc/ssl/certs/**
   - **sudo chown root:ssl-cert /etc/ssl/private/seguro.key**
   - **sudo chmod 640 /etc/ssl/private/seguro.key**
   - **sudo chown root:root /etc/ssl/certs/seguro.crt**

9. Crea el fichero **/etc/apache/sites-available/seguro.conf** con las siguientes directivas:

```
<IfModule mod_ssl.c>
        <VirtualHost *:443>
                ServerName seguro.dawXX.net
                DocumentRoot  /var/www/html/seguro
                ErrorLog ${APACHE_LOG_DIR}/seguro_error.log
                CustomLog ${APACHE_LOG_DIR}/seguro_access.log combined
                <Directory /var/www/html/seguro>
                        Options Indexes FollowSymLinks
                        AllowOverride None
                        Require all granted
                </Directory>
                SSLEngine on
                SSLCertificateFile /etc/ssl/certs/seguro.crt
                SSLCertificateKeyFile  /etc/ssl/private/seguro.key
    </VirtualHost>
</IfModule>
```

10. Deshabilita el servidor ssl por defecto.

    sudo a2dissite default-ssl

11. Habilita el servidor virtual seguro.

    sudo a2ensite seguro

    Observa que dentro del directorio **/etc/apache2/sites-enabled** se ha creado el enlace **seguro**.

12. Finalmente, podemos establecer la conexión desde un navegador:

    https://seguro.com